

Modellazione e simulazione a eventi discreti di una pipeline CI/CD

Una rete di code $M^{[B]}/G/c$ con feedback umano, calibrata e validata su TravisTorrent

Andrea Cantarini (0351377)

Corso di Performance Modeling of Computer Systems and Networks
Università degli Studi di Roma “Tor Vergata”

Roma, Italia

andrea.cantarini@alumni.uniroma2.eu

Sommario—Si presenta uno studio di modellazione e simulazione a eventi discreti della piattaforma di Continuous Integration Travis CI, formalizzata come rete di code $M^{[B]}/G/c$ con arrivi a gruppi e anello di retroazione umana. Ogni parametro del modello è stimato da tracce di esecuzione reali estratte dal dataset TravisTorrent e relative al progetto SonarSource/sonar-java (16278 job su 476 giorni). Il simulatore, scritto in Python su un generatore di Lehmer multi-stream, è sottoposto a una verifica articolata su cinque livelli — validità dei valori, legge di Little, generatore pseudo-casuale, generatori di variate e confronto con la soluzione analitica esatta di una $M^{[B]}/G/1$ su cinque livelli di utilizzazione, quest’ultimo completato dai controlli di conservazione del flusso — per un totale di 70 controlli automatici, tutti superati. La validazione confronta le grandezze misurate dalle tracce simulate con quelle misurate sul dataset reale, ottenendo scarti non superiori a $2 \cdot 10^{-4}$ sulle probabilità di esito e inferiori allo 0,4% sul tasso di arrivo; i residui sui tempi di servizio sono scomposti analiticamente nelle loro cause. Lo studio what-if sul numero di serventi mostra che il sistema opera a utilizzazione $\rho \approx 0,03$ e che il tempo di attesa non è generato dal carico ma dalla serializzazione dei job all’interno della stessa build matrix: la teoria classica $M/M/c$ sottostima l’attesa osservata di diversi ordini di grandezza. Il collo di bottiglia effettivo risulta essere il ritardo di revisione umana, che impone al tempo di risposta un limite inferiore di circa 850 s indipendente dal dimensionamento dell’infrastruttura.

Index Terms—simulazione a eventi discreti, reti di code, arrivi a gruppi, continuous integration, verifica e validazione, batch means

I. SISTEMA OGGETTO DI STUDIO

Travis CI¹ è una piattaforma cloud dedicata alla Continuous Integration e al Continuous Deployment (CI/CD), a lungo fra le più adottate nell’ambito dei progetti open-source. Il suo scopo è automatizzare l’integrazione e il collaudo del codice: attraverso un sistema di pipeline la piattaforma scarica il codice, lo compila ed esegue i test in maniera automatizzata e configurabile, così da garantire che le nuove modifiche non introducano difetti o conflitti nel software esistente.

La pipeline è definita in modo dichiarativo tramite un file di configurazione (`.travis.yml`) collocato nella root del progetto. Ogni volta che si verifica un evento di interesse — tipicamente un commit o l’apertura di una pull request

— Travis CI innesca una *build* su un server dedicato, detto *runner*.

Per ridurre i tempi e massimizzare la copertura dei test, una singola build viene tipicamente suddivisa in molteplici unità di lavoro atomiche, chiamate *job*. I progetti utilizzano di norma una *build matrix* per lanciare più job in parallelo, verificando il medesimo codice su versioni diverse di un linguaggio o su sistemi operativi differenti. È questa la struttura che, sul piano modellistico, si traduce in un processo di arrivo *a gruppi*: non arriva un job alla volta, ma un intero grappolo di job simultanei.

Storicamente, per i progetti open-source Travis metteva a disposizione cinque runner per repository. A partire dal 2020, in seguito all’acquisizione da parte di Idera Inc. e alla migrazione del dominio da `travis-ci.org` a `travis-ci.com`, il supporto gratuito per i progetti open-source è stato terminato². Queste modifiche hanno spinto gran parte dei progetti open-source a migrare verso altre piattaforme di CI/CD, prima fra tutte GitHub Actions. Il dimensionamento del pool di runner ha dunque assunto, per chi ha scelto di restare sulla piattaforma, una rilevanza economica diretta: ogni runner aggiuntivo ha un costo, e occorre sapere quale beneficio prestazionale esso produca.

L’impatto di Travis CI nei suoi anni di massima diffusione è documentato nel dataset TravisTorrent³, che raccoglie i dati di circa quattro milioni di job, eseguiti su quasi mille progetti open-source ospitati su GitHub nel periodo compreso fra il 2011 e il 2016. È da questa fonte che il presente studio trae tanto i parametri del modello quanto il riferimento oggettivo per la sua validazione.

II. OBIETTIVI DELLO STUDIO

Lo studio si pone un duplice obiettivo:

- 1) **Caratterizzare** le prestazioni di Travis CI nella configurazione corrispondente al piano gratuito storicamente offerto ai progetti open-source ($c = 5$ runner), quantificando tempo di risposta, tempo di attesa, utilizzazione e popolazioni medie in condizioni di regime.

²<https://travis-ci.community/t/org-com-migration-unexpectedly-comes-with-a-plan-change>

³<https://figshare.com/articles/dataset/TravisTorrent/19314170>

¹<https://www.travis-ci.com/>

2) **Stimare** come tali prestazioni varino al variare del numero di runner a disposizione della repository (*what-if analysis*), così da fornire un criterio quantitativo per la scelta del piano di servizio.

A questi due obiettivi di natura prestazionale se ne affianca un terzo, di natura metodologica: **stabilire in modo verificabile che il modello computazionale sia corretto** (verifica) e **che esso riproduca il comportamento del sistema reale** (validazione). Gran parte del presente documento è dedicata a questo terzo obiettivo, perché è da esso che dipende l'attendibilità delle conclusioni degli altri due.

A. Progettazione degli esperimenti

L'esperimento di base consiste nell'implementazione del modello così come descritto nelle Sezioni IV–VI, nella configurazione a $c = 5$ serventi. Su di essa si eseguono l'analisi del transitorio, l'analisi del comportamento a regime (*steady-state*), la verifica e la validazione.

Successivamente si varia il numero di serventi, eseguendo la simulazione con $c \in \{1, 2, 5, 10\}$ runner e lasciando invariati tutti gli altri parametri. I quattro valori sono scelti per coprire l'intervallo di interesse pratico: $c = 1$ rappresenta il piano minimo, $c = 2$ un piano intermedio, $c = 5$ la configurazione storica per l'open-source e $c = 10$ un piano sovradimensionato pari alla massima dimensione di build matrix osservata nel progetto.

Per ciascuna configurazione si conducono due analisi complementari:

- un'analisi a **orizzonte finito** con il metodo delle *repliche indipendenti* (cinque semi distinti), volta a studiare il transitorio e la convergenza al regime;
- un'analisi a **orizzonte infinito** con il metodo dei *batch means* (singola run lunga divisa in 64 batch dopo l'eliminazione del transitorio), volta a stimare le grandezze di regime con i relativi intervalli di confidenza.

III. ANALISI DEL DATASET E CALIBRAZIONE DEI PARAMETRI

Ogni parametro del modello è stato stimato a partire da tracce di esecuzione reali. Questa sezione documenta il percorso che porta dal dataset grezzo ai valori numerici impiegati nella simulazione, motivando per ciascuna grandezza la scelta della famiglia distributiva e il metodo di stima.

La fase di analisi persegue tre finalità distinte: (i) selezionare un progetto reale che presenti tutte le caratteristiche strutturali richieste dal modello; (ii) stimare i parametri stocastici del sistema; (iii) predisporre il riferimento rispetto a cui, a valle della simulazione, verranno confrontate le grandezze misurate dalle tracce prodotte dal simulatore.

Il risultato finale confluisce interamente in un file di configurazione esterno (`config.json`): **nessun parametro del modello è scritto nel codice sorgente**, così che la calibrazione resti separata, ispezionabile e modificabile senza toccare il motore di simulazione.

Tabella I
COLONNE DEL DATASET TRAVISTORRENT IMPIEGATE NELLA CALIBRAZIONE

Colonna	Ruolo nella calibrazione
gh_project_name	Chiave di selezione del progetto
tr_build_id	Raggruppa i job in gruppi; individua l'istante di arrivo
tr_job_id	Unità servita dal modello; conteggio della dimensione del gruppo
git_trigger_commit	Chiave del feedback loop: lega fra loro i tentativi ripetuti sullo stesso commit
gh_build_started_at	Base dell'intero dominio temporale: inter-arrivi, fine job, intervalli di feedback
tr_status	Probabilità di instradamento; segmentazione dei tempi di servizio
tr_log_setup_time	Prima fase del servizio
tr_log_buildduration	Seconda fase del servizio (al netto del setup); istante di fine job

A. Fonte dei dati e pipeline di estrazione

Si è utilizzata la release `final-2017-01-25` di TravisTorrent [2], che congiunge i log di build di Travis CI ai metadati GitHub dei rispettivi repository. Il file grezzo, di circa 250 MB compressi, contiene 66 colonne e alcuni milioni di record, dove **ogni riga corrisponde a un singolo job** (una cella della build matrix) e non a una build intera.

Data la dimensione, tutte le estrazioni sono state effettuate con la libreria Polars in modalità *lazy* (`scan_csv` seguito da `collect()`): il piano di esecuzione viene ottimizzato e il filtro sul progetto viene spinto a monte, così che in memoria arrivino soltanto i record effettivamente necessari.

Due accorgimenti di lettura si sono rivelati necessari e vale la pena documentarli, perché la loro omissione produce silenziosamente colonne nulle:

- l'opzione `null_values=["NA", ""]`, perché il dataset codifica i valori mancanti come stringa letterale e non come campo vuoto;
- la forzatura a stringa delle colonne `tr_log_setup_time` e `tr_log_buildduration`, seguita da un cast esplicito a `Float64` in modalità non stretta. L'inferenza automatica del tipo su queste due colonne è infatti inaffidabile per la presenza di valori misti: la finestra iniziale suggerisce il tipo intero e il parsing fallisce poi sui valori decimali.

1) *Colonne impiegate*: Delle 66 colonne disponibili ne sono state impiegate otto, elencate nella Tabella I. Da esse sono state derivate per calcolo le grandezze di lavoro: istante di inizio e fine job, durata netta della fase di test, istante di inizio e fine build, conteggio dei job difettosi per build, indicatore di nuovo arrivo, indicatore di riavvio, tempo di feedback e numero di job per build.

B. Selezione del caso di studio

Il modello richiede un caso di studio che esibisca simultaneamente arrivi a gruppi di dimensione variabile, tutti e quattro gli esiti possibili, tempi registrati in modo sufficientemente

Tabella II
SCREENING DEI PROGETTI CANDIDATI

Progetto	Job	% fail	% err	max B	σ_B
checkstyle/checkstyle	46 889	7,10	0,77	35	10,14
xetorthio/jedis	3 258	13,75	2,12	5	0,50
SonarSource/sonar-java	16 278	3,45	18,03	10	2,33
geoserver/geoserver	6 742	46,72	6,08	6	1,08

completo e una casistica adeguata di riavvii. Si è pertanto costruito uno screening quantitativo su tutti i progetti del dataset, basato su quattro filtri:

- **numerosità:** almeno 1000 job, per garantire stime statisticamente significative;
- **completezza:** maggior numero possibile di valori non nulli sulle cinque colonne critiche (quelle riportate in Tabella I, escludendo i tre identificativi), per evitare stime su dati largamente mancanti;
- **variabilità del gruppo:** almeno 5 job nella build più grande, poiché un modello $M^{[B]}$ richiede un gruppo non degenero;
- **presenza degli esiti:** percentuali di failed, errored e canceled tutte strettamente positive, poiché servono tutti i rami di instradamento, inclusi i riavvii.

I candidati superstiti sono stati ordinati per completezza e per deviazione standard del numero di job per build; i primi risultati sono riportati nella Tabella II.

La scelta è caduta su SonarSource/sonar-java. Le ragioni sono di merito modellistico più che di semplice graduatoria:

- checkstyle/checkstyle, pur essendo il più ricco in termini di volume, presenta una percentuale di errored trascurabile (0,77%) e una dispersione del gruppo anomala ($\sigma_B = 10,14$, massimo 35 job), indice di una build matrix disomogenea nel tempo che comprometterebbe l'ipotesi di distribuzione stazionaria della dimensione del gruppo;
- xetorthio/jedis ha gruppi quasi deterministici ($\sigma_B = 0,50$), il che renderebbe superflua la componente $M^{[B]}$ del modello;
- sonar-java offre invece un insieme di esiti ben distribuito su tutti e quattro i rami, una dimensione di gruppo genuinamente variabile ma limitata (da 1 a 10 job) e un volume adeguato.

Il campione finale è composto da **16 278 job**, raggruppati in **2 278 build**, generate da **1 176 commit distinti**, su un arco temporale osservato di **476,8 giorni** (circa 15,7 mesi), pari mediamente a circa 4 build al giorno. Il sottoinsieme è stato estratto una sola volta e salvato su file, così da costituire un riferimento stabile per la fase di validazione.

C. Separazione fra traffico esogeno ed endogeno

Il punto metodologicamente più delicato dell'intera analisi riguarda il processo di arrivo. Nel dataset, una build che riparte dopo un fallimento è indistinguibile, a livello di riga, da una

build nuova. Se si contassero ingenuamente tutte le build come arrivi, si conteggerebbe **due volte** il traffico generato dal feedback loop, che il modello genera già internamente: il tasso di arrivo risulterebbe sovrastimato e il sistema apparirebbe più carico di quanto sia.

Si è quindi definito un criterio esplicito di separazione fra traffico esogeno ed endogeno:

*Una build è un **nuovo arrivo** se è la prima build del suo `git_trigger_commit`, oppure se la build precedente sullo stesso commit era “buona” (priva di job failed o errored). Altrimenti è un **riavvio**, cioè traffico generato dal feedback loop.*

Applicando il filtro, delle 2 278 build totali ne restano **1 903 come arrivi esogeni**: il 16% delle build osservate è dunque traffico di ritorno e non traffico nuovo.

D. Processo di arrivo

Il fit esponenziale sugli inter-arrivi così filtrati fornisce un tempo medio di inter-arrivo di **21 820 s** (circa 6 ore), da cui $\lambda = 4,6 \cdot 10^{-5}$ arrivi al secondo, valore adottato nel modello. Poiché λ è arrotondato a due cifre significative, il modello realizza in realtà un inter-arrivo medio di $1/\lambda = 21 739$ s, inferiore dello 0,37% alla stima: è questo il valore che compare come momento teorico nella verifica dei generatori di variare (Tabella VII).

a) *Adeguatezza dell'ipotesi poissoniana:* Il test di Kolmogorov–Smirnov rifiuta nettamente l'esponenziale ($D = 0,370$). Il motivo è evidente dalle statistiche descrittive: la mediana vale 3 228 s contro una media di 21 820 s, e il coefficiente di variazione è $CV = 2,14$ a fronte del valore unitario richiesto da una esponenziale. Il processo reale è marcatamente *burst-oriented*: il 21% degli inter-arrivi è inferiore ai 10 minuti, riflesso di sessioni di lavoro intense dello sviluppatore, mentre la coda si estende fino a 5,9 giorni, riflesso di pause, fine settimana e ferie.

L'ipotesi di arrivi poissoniani è stata nondimeno **mantenuta consapevolmente**, per tre ragioni: preserva la trattabilità analitica del confronto con i modelli di riferimento, è l'assunzione standard per questa classe di modelli e, soprattutto, la sovradisersione osservata agisce in direzione ottimistica dato che il sistema opera molto lontano dalla saturazione (cfr. §III-I). Il punto è ripreso fra i limiti dello studio in §III-J.

E. Dimensione dei gruppi

Ogni commit scatena una build matrix di più job paralleli. Contando i job per build sulle 2 278 build si ottiene la distribuzione di probabilità empirica riportata nella Tabella III, con valore atteso $E[B] = 7,145$ job per build e $E[B^2] = 56,50$.

Due caratteristiche giustificano l'adozione di una distribuzione empirica anziché di una famiglia parametrica. La prima è la **forte multimodalità**, con un picco dominante a 8 job che raccoglie da solo il 34,7% della massa. La seconda è la presenza di un **bucco strutturale a 5**, con frequenza esattamente nulla, riflesso della particolare configurazione della matrice di build del progetto. Nessuna distribuzione discreta standard

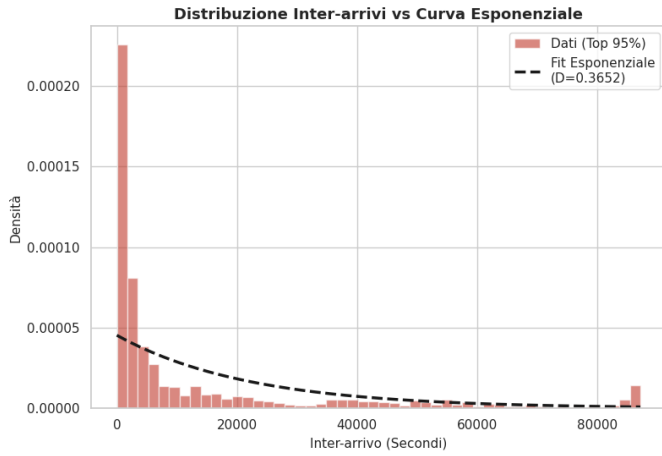


Figura 1. Distribuzione empirica dei tempi di inter-arrivo dei soli arrivi esogeni e densità esponenziale di parametro $\lambda = 4,6 \cdot 10^{-5} \text{ j/s}$

Tabella III
DISTRIBUZIONE DI PROBABILITÀ DELLA DIMENSIONE DEL GRUPPO (JOB PER BUILD)

Job per build	Frequenza	Probabilità
1	97	,043
2	43	,019
3	104	,046
4	172	,076
6	241	,106
7	178	,078
8	790	,347
9	447	,196
10	206	,090

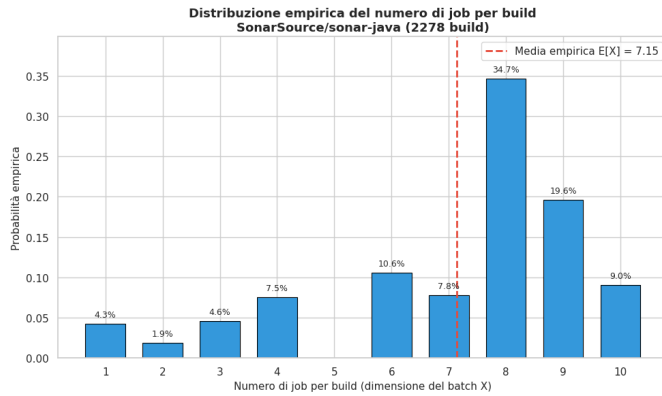


Figura 2. Distribuzione empirica del numero di job per build nel progetto sonar-java. Si osservano la moda di 8 job e l'assenza completa di build da 5 job.

riprodurrebbe questa forma, e forzarne una peggiorerebbe il modello anziché migliorarlo.

F. Probabilità di instradamento

Le probabilità di esito sono calcolate sulla colonna `tr_status` di **tutti** i 16278 job. Si è deliberatamente evitata qualsiasi rimozione di righe per valori mancanti su colonne non pertinenti, quali timestamp o commit: scartare record per

mancanze estranee altererebbe i denominatori e distorcerebbe le probabilità. I valori ottenuti sono $p_{\text{passed}} = 0,773436$, $p_{\text{errored}} = 0,180305$, $p_{\text{failed}} = 0,034525$ e $p_{\text{canceled}} = 0,011734$.

Nel simulatore l'esito è estratto con un meccanismo a **rischi competitivi**: una singola variabile uniforme in $(0,1)$ determina il destino del job, ossia lo stato con cui terminerà. Le etichette sono allineate per nome fra configurazione e dataset, così che il confronto in fase di validazione risulti diretto e non ambiguo.

G. Tempi di servizio

Il servizio di un job è modellato in due fasi consecutive: il **setup**, ossia la preparazione dell'ambiente e lo scaricamento delle dipendenze, e la fase di **test** vera e propria. La seconda si ottiene per differenza fra la durata totale registrata e il tempo di setup.

1) *Fase di setup*: Su 15426 job con setup valido si osservano una media di 2,02 s, una mediana di 2 s, un massimo di 94 s e soltanto **28 valori distinti**. Il tentativo di fit lognormale fallisce ($D = 0,375$). Infatti, il setup non è una variabile continua, ma un valore **discreto** concentrato su pochi interi (49,1% a 1 s, 26,7% a 2 s, 11,5% a 3 s, 9,2% a 4 s, per oltre il 96% dei campioni in quattro soli valori), con una manciata di valori anomali di origine infrastrutturale. Si è dunque adottata direttamente la **distribuzione empirica** a 28 valori. È un caso esemplare in cui forzare una distribuzione teorica avrebbe peggiorato il modello.

2) *Fase di test dei job passed*: La seconda fase è stimata sui soli job con esito `passed`, che rappresentano l'esecuzione "ideale" priva di interruzioni anticipate, e sulla durata *netta* del setup. Su 9078 osservazioni il fit lognormale a massima verosimiglianza (con parametro di posizione vincolato a zero, per rispettare il supporto positivo) fornisce $\mu = 5,9545$ e $\sigma = 0,5316$, contro una media empirica di 440,1 s, con $D = 0,0307$.

La statistica D è piccola in valore assoluto e la sovrapposizione grafica fra istogramma e densità teorica è ottima (Fig. 3). Formalmente, il test rifiuta comunque l'ipotesi nulla, ma si tratta di un esito atteso: con $n \approx 9000$ il test di Kolmogorov-Smirnov diventa sensibilissimo a scostamenti irrilevanti sul piano pratico. Si decide dunque di procedere per **praticità** anziché per non-rifiuto formale. Si noti fin d'ora che la media implicata dalla lognormale stimata vale $e^{\mu+\sigma^2/2} = 443,99$ s, superiore di 3,91 s alla media empirica: questa differenza sarà discussa in fase di validazione del modello (§VIII-C).

3) *Job failed e canceled*: Questi esiti non sono stati modellati come troncamenti casuali del servizio nominale. I dati mostrano infatti che un job `failed` esegue quasi per intero la build prima di fallire, con una media di 382,2 s contro i 439,5 s di un `passed`: interpretarlo come un troncamento a metà del servizio introdurrebbe una sottostima sistematica del carico. Sono stati perciò calibrati su lognormali dedicate, stimate sulle rispettive durate reali.

La stima **non** impiega la massima verosimiglianza bensì il **metodo dei momenti**, e la ragione è sostanziale. Per una lognormale la massima verosimiglianza coincide con la stima normale sui logaritmi, $\hat{\mu} = \log x$ e $\hat{\sigma}^2 = \text{Var}(\log x)$: il fit

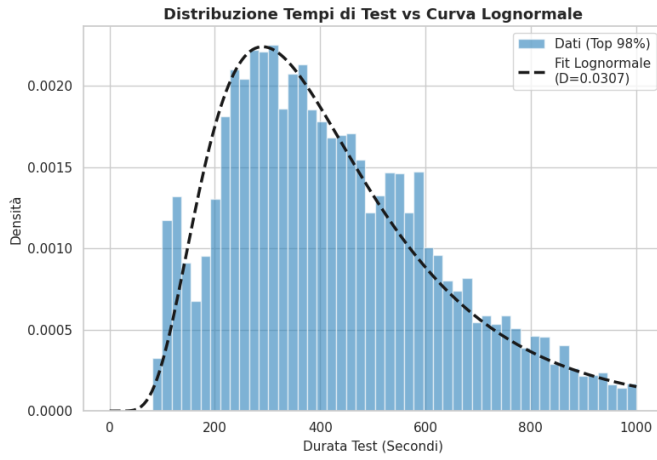


Figura 3. Durata della fase di test dei job *passed* (al netto del setup) e densità lognormale di parametri $\mu = 5,9545$, $\sigma = 0,5316$ stimata a massima verosimiglianza.

è dunque condotto *in scala logaritmica*, dove il criterio di aderenza è la mediana. La grandezza che governa il comportamento di un modello di code è però la domanda di servizio attesa,

$$E[S] = e^{\mu + \sigma^2/2}, \quad (1)$$

nella quale σ entra esponenziata al quadrato. Una lieve imprecisione sulla dispersione logaritmica si traduce perciò in un errore moltiplicativo sulla media, ed è esattamente quanto accade qui: la massima verosimiglianza restituisce $\hat{\sigma} = 1,0891$ per i *failed* e $\hat{\sigma} = 1,4593$ per i *canceled*, cui corrispondono medie campionarie di 484,2 s e 593,6 s contro le 380,7 s e 330,6 s effettivamente osservate — una sovrastima del **27%** e dell'**80%**.

La causa dello scarto non è una coda destra più pesante di quella lognormale: il coefficiente di variazione empirico, $CV = \sqrt{\text{Var}[S]}/E[S]$, vale 0,653 e 0,680, mentre le lognormali stimate per massima verosimiglianza ne implicano rispettivamente 1,51 e 2,72.⁴ I dati sono cioè *meno* dispersi, non più dispersi, di quanto la massima verosimiglianza suggerisca. Il fenomeno è piuttosto una **contaminazione all'estremo inferiore**: l'8,1% dei *failed* e il 10,0% dei *canceled* dura meno di sessanta secondi, con minimi di 3 s e 1 s, a fronte di mediane di 336 s e 328 s. In scala lineare queste osservazioni sono irrilevanti per la media; in scala logaritmica distano quattro o cinque unità dal corpo della distribuzione e, entrando al quadrato nella varianza, ne determinano da sole la stima. L'asimmetria dei logaritmi lo conferma: $-1,88$ e $-2,42$, dove una lognormale esatta darebbe zero. La verifica è immediata — escludendo le sole durate inferiori a trenta

⁴Si impiegano nel seguito entrambe le misure adimensionali di dispersione, secondo il contesto: il *coefficiente di variazione* $CV = \sigma/\mu$ dove il discorso riguarda l'errore standard di una stima, che scala linearmente con esso; il *coefficiente di variabilità* $C^2 = \sigma^2/\mu^2 = CV^2$ dove il discorso riguarda il comportamento del sistema a coda, poiché è questa la grandezza che compare linearmente nelle formule di attesa e nelle scomposizioni della varianza.

secondi, $\hat{\sigma}$ scende a 0,649 e 0,641, praticamente sui valori dei momenti, e la sovrastima della media crolla al 2,8% e al 2,6%.

Si tratta dunque della coesistenza di due popolazioni distinte: le build effettivamente eseguite e quelle interrotte quasi subito. È lo stesso fenomeno che per gli *errored* viene modellato in modo esplicito attraverso $p_{\text{errored,early}}$; per *failed* e *canceled*, la cui numerosità non consentirebbe di stimare in modo affidabile i parametri di due modi separati, lo si assorbe invece in un'unica lognormale. Proprio per questo si sono scelti μ e σ che **eguagliano per costruzione media e varianza empiriche**,

$$\sigma^2 = \ln(1 + C^2), \quad \mu = \ln E[S] - \sigma^2/2, \quad (2)$$

ottenendo $\mu = 5,7644$, $\sigma = 0,5958$ per i *failed* e $\mu = 5,6105$, $\sigma = 0,6168$ per i *canceled*. La scelta sacrifica l'aderenza nel corpo della distribuzione — che la (2) non controlla — in cambio della correttezza esatta del primo e del secondo momento, che sono le uniche due grandezze da cui dipendono l'utilizzazione e, attraverso la formula di Pollaczek–Khinchine, l'attesa in coda. Per lo scopo del modello è il compromesso corretto.

4) *Job errored: struttura bimodale*: L'analisi dei 2935 job *errored* ha rivelato una struttura a due popolazioni nettamente distinte:

- **2 543 job (86,64%)** non hanno alcuna durata registrata. Si tratta di rotture **precoci**: il job muore durante il setup, prima che la fase di test inizi, per errore infrastrutturale, dipendenze irrisolte o timeout di rete;
- **392 job (13,36%)** hanno una durata regolare, con media 432,1 s, praticamente identica a quella di un *passed*. Sono job che hanno eseguito l'intera build e sono andati in errore alla fine.

Trattare uniformemente le due popolazioni — assegnando a tutti gli *errored* servizio nullo, oppure a tutti il servizio pieno — introdurrebbe un errore sistematico rilevante sul carico, dato che gli *errored* rappresentano il 18% dei job. Il simulatore riproduce quindi la bimodalità: con probabilità $p_{\text{errored,early}} = 0,8664$ il servizio è una frazione uniforme del solo setup, altrimenti è pari a quello di un *passed*.

H. Ritardo di feedback umano e probabilità di riavvio

Il nodo di feedback modella il tempo di reazione dello sviluppatore di fronte a una build fallita. Non essendo osservabile direttamente, esso è stato ricostruito come intervallo temporale fra la fine di una build “cattiva” e l'inizio della build successiva sullo stesso commit, dove la fine della build è stimata dal job più lungo che la compone e le durate mancanti sono trattate come nulle, coerentemente con il modello di errore precoce.

Il filtro è vincolato ai soli intervalli che partono da una build *effettivamente fallita*, ossia con almeno un job *failed* o *errored*: un riavvio dopo una build riuscita non è una reazione a un fallimento e inquinerebbe la stima. Sono inoltre scartati gli intervalli superiori a **48 ore**, che non descrivono più una reazione a un fallimento ma la ripresa del lavoro dopo una interruzione prolungata: il taglio rimuove una sola osservazione, e sarà ripreso più avanti perché la sua influenza

sulla media campionaria è sproporzionata al suo peso. Si ottengono così **327 eventi di riavvio validi**.

Il fit lognormale è effettuato sui tempi espressi in **minuti**, coerentemente con l'interpretazione del simulatore, e fornisce $\mu_{\text{delay}} = 0,9591$ e $\sigma_{\text{delay}} = 2,1399$, da cui una mediana di circa 2,6 minuti e una media teorica di circa 25,8 minuti.

Il valore molto elevato di σ merita un commento esplicito, perché condiziona pesantemente i risultati e la precisione delle stime. La distribuzione è **estremamente dispersa**, con mediana empirica di 1,4 minuti contro una media empirica di 87,8 minuti: il coefficiente di variazione teorico vale $\sqrt{e^{\sigma^2} - 1} \approx 9,8$, cui corrisponde un coefficiente di variabilità $C_D^2 = e^{\sigma^2} - 1 \approx 96,4$. Questo cattura un comportamento reale e ben noto — la maggior parte dei riavvii è quasi immediata, perché lo sviluppatore sta osservando il cruscotto e rilancia subito, ma una minoranza avviene ore dopo. La lognormale con σ grande è la scelta naturale per una dispersione di questo ordine; si vedrà in §XI-F che essa è anche la causa dominante dell'ampiezza degli intervalli di confidenza sul tempo di risposta.

a) *Perché qui la massima verosimiglianza e non il metodo dei momenti*: La scelta dell'estimatore va dichiarata, perché è l'opposto di quella compiuta poco sopra per *failed* e *canceled*, dove la massima verosimiglianza è stata scartata proprio in quanto sbagliava la media. Qui il divario è persino più ampio, e di segno contrario: la lognormale stimata implica $E[D] = 25,8$ minuti a fronte di una media campionaria di 87,8, ossia una sottostima di un **fattore 3,4**. Applicando anche a questi dati la (2) si otterrebbero $\mu_{\text{delay}} \approx 3,17$ e $\sigma_{\text{delay}} \approx 1,61$.

Le due situazioni sono però strutturalmente diverse. Per *failed* e *canceled* lo scarto nasceva all'estremo *inferiore* della distribuzione e, rimossa la contaminazione, la massima verosimiglianza convergeva ai valori dei momenti: questi ultimi erano dunque la stima "pulita". Qui lo scarto nasce all'estremo *superiore*, e la media campionaria non è una statistica affidabile. I dieci intervalli più lunghi contribuiscono da soli circa il **54%** della somma complessiva, i venti più lunghi oltre l'80%, e il solo massimo quasi il 7%; l'errore standard della media vale il **19,5%**, e la troncatura a 48 ore descritta sopra — che rimuove una singola osservazione su 328 — sposta già la media da 96,5 a 87,8 minuti. Calibrare sul primo momento significherebbe far dipendere il parametro più influente del modello da una decina di osservazioni.

Il costo sul corpo della distribuzione sarebbe inoltre proibitivo. La lognormale dei momenti avrebbe mediana 23,9 minuti contro 1,4 osservata, e collocherebbe sotto i cinque minuti appena il 16,6% dei riavvii a fronte di un 77,4% empirico; la stima adottata, con mediana 2,6 minuti, ne colloca sotto i cinque minuti il 61,9%. Nessuna delle due riproduce esattamente il corpo empirico, ma la massima verosimiglianza sbaglia di un fattore 1,25 dove il metodo dei momenti sbaglia di un fattore 4,7; coerentemente, la statistica di Kolmogorov–Smirnov peggiorerebbe da $D = 0,214$ a $D = 0,628$. Il criterio che ha guidato l'intera calibrazione — preservare la grandezza da cui dipende il comportamento del sistema — conduce qui alla conclusione opposta rispetto ai tempi di servizio, e per

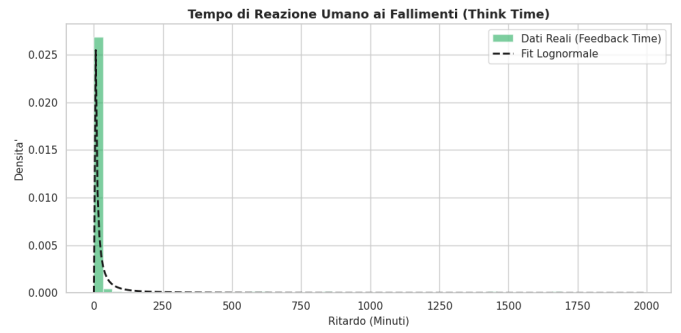


Figura 4. Distribuzione dei 327 intervalli di riavvio osservati e densità lognormale stimata ($\mu = 0,9591$, $\sigma = 2,1399$, tempi in minuti). La scala logaritmica in ascissa rende leggibile una distribuzione che si estende su oltre quattro ordini di grandezza.

una ragione precisa: il nodo di feedback ha infiniti serventi, non possiede coda, e la sua variabilità non entra in alcuna formula di attesa. Ciò che se ne osserva nel tempo di risposta è la distribuzione nella sua interezza, non i suoi primi due momenti, ed è quindi il corpo della distribuzione che occorre riprodurre.

Resta il fatto che la scelta **sottostima** il ritardo di revisione, ed è bene dichiarare che essa opera *contro* la tesi sostenuta in §XI. Adottando i momenti, la componente di feedback del tempo di risposta minimo passerebbe da 398,7 a 1359 s, il tempo di risposta minimo da 846,6 a 1807 s e la quota attribuibile alla revisione umana dal 47,1% al 75,2%. La conclusione sul collo di bottiglia vale dunque *a fortiori* sotto la stima alternativa, e non può essere imputata alla scelta dell'estimatore.

1) *Probabilità di riavvio*: Dopo il ritardo umano il job rientra in coda soltanto con una certa probabilità; altrimenti abbandona il sistema, perché il commit viene abbandonato o corretto altrove. Un job è classificato come *effettivamente riavviato* se sullo stesso `git_trigger_commit` esiste un avvio successivo alla morte del job stesso. Si ottengono $P_{\text{retry,failed}} = 0,3327$ (187 riavvii su 562 job) e $P_{\text{retry,errorred}} = 0,8637$ (2535 su 2935).

Il contrasto fra i due valori è netto e interpretabile: un *errored* è tipicamente un guasto infrastrutturale transitorio, che lo sviluppatore riprova quasi sempre; un *failed* segnala invece un difetto reale nel codice, che richiede una correzione e più spesso porta ad abbandonare quel tentativo. Anche in questo calcolo si è evitato di richiedere la presenza della durata di build: farlo avrebbe scartato l'87% dei job *errored* — proprio quelli a rottura precoce — con una distorsione massiccia della stima.

I. Quadro riassuntivo e carico implicato

La Tabella IV riassume tutti i parametri stimati, con l'indicazione della loro origine metodologica.

Dai parametri calibrati discende direttamente il regime operativo del sistema. Il tasso di arrivo dei job esogeni vale

Tabella IV
QUADRO RIASSUNTIVO DEI PARAMETRI DEL MODELLO E LORO ORIGINE

Sezione	Parametro	Valore	Metodo di stima
Traffico	λ	$4,6 \cdot 10^{-5} \text{ s}^{-1}$	Fit esponenziale sui soli arrivi esogeni (§III-C)
Traffico	distribuzione di B	9 valori, $E[B] = 7,145$	Distribuzione empirica dei job per build
Instradamento	$p_{\text{passed}} / p_{\text{failed}} / p_{\text{errored}} / p_{\text{canceled}}$	0,7734 / 0,0345 / 0,1803 / 0,0117	Frequenze relative di <code>tr_status</code> su tutti i job
Instradamento	$p_{\text{errored,early}}$	0,8664	Frazione di <code>errored</code> priva di durata registrata
Servizio, setup	distribuzione di S_{setup}	28 valori, media 2,02 s	Distribuzione empirica (fit continuo rifiutato, $D = 0,375$)
Servizio, test	μ / σ	5,9545 / 0,5316	Lognormale, massima verosimiglianza sui <code>passed</code>
Servizio, failed	μ / σ	5,7644 / 0,5958	Lognormale, metodo dei momenti
Servizio, canceled	μ / σ	5,6105 / 0,6168	Lognormale, metodo dei momenti
Feedback umano	$\mu_{\text{delay}} / \sigma_{\text{delay}}$	0,9591 / 2,1399 (minuti)	Lognormale sugli intervalli di riavvio
Feedback umano	$p_{\text{retry,failed}} / p_{\text{retry,errored}}$	0,3327 / 0,8637	Frequenze di riavvio effettivamente osservate

$\lambda E[B] = 3,29 \cdot 10^{-4} \text{ job/s}$. La probabilità che un job rientri nel loop è

$$q = p_{\text{failed}} p_{\text{retry,failed}} + p_{\text{errored}} p_{\text{retry,errored}} = 0,16722, \quad (3)$$

da cui un fattore di visita

$$v = \frac{1}{1 - q} = 1,20079 \quad (4)$$

e un tasso di arrivo complessivo al nodo di servizio pari a $\lambda_{\text{nodo}} = \lambda E[B]v = 3,947 \cdot 10^{-4} \text{ job/s}$. Il tempo medio di servizio, mediato sui quattro esiti secondo le rispettive probabilità e comprensivo del setup, vale $E[S] = 372,98 \text{ s}$.

Ne segue, con la configurazione nominale a $c = 5$ serventi, un'utilizzazione

$$\rho = \frac{\lambda_{\text{nodo}} E[S]}{c} = 0,0294. \quad (5)$$

Si noti che le relazioni (3)-(5) sono equazioni di bilancio del flusso e non una soluzione del modello: esse determinano throughput e utilizzazione, che in un sistema stabile e senza perdite non dipendono dalla disciplina di servizio, ma nulla dicono sui tempi di attesa. Per la $M^{[B]}/G/c$ con $c > 1$ e feedback non poissoniano questi ultimi non ammettono forma chiusa, ed è precisamente questa la ragione del ricorso alla simulazione.

Il sistema reale opera dunque **molto lontano dalla saturazione**: anche con un solo servente si avrebbe $\rho \approx 0,147$. Questa è una conclusione importante della fase di analisi e non un difetto della calibrazione: con circa quattro build al giorno e job da poco più di sei minuti, l'infrastruttura di integrazione continua di `sonar-java` è largamente sovradimensionata rispetto al carico medio. Se ne trarranno le conseguenze nella discussione dei risultati (§XI), dove si mostrerà che, ciò nonostante, l'attesa in coda *non* è trascurabile — e che la ragione va cercata altrove che nel carico.

J. Assunzioni e limiti della calibrazione

Si riportano esplicitamente le scelte che condizionano la validità dei risultati, con l'indicazione, ove possibile, della direzione dell'errore introdotto.

- 1) **Arrivi poissoniani.** L'ipotesi è rifiutata dal test di Kolmogorov–Smirnov ($D = 0,370$, $CV = 2,14$ contro 1 atteso): il processo reale è a raffiche, con cicli

giorno/notte e settimanali che il modello omogeneo non riproduce. La conseguenza è una sottostima dei picchi di coda istantanei; l'effetto è tuttavia attenuato dal bassissimo fattore di utilizzazione.

- 2) **Separazione esogeno/endogeno euristica.** Il criterio basato sul commit di innesco è ragionevole ma non perfetto: un riavvio manuale su una build riuscita viene classificato come nuovo arrivo. Vi è una lieve sovrastima del traffico esogeno.
- 3) **Istante di fine build stimato.** La fine di una build è approssimata con il job più lungo, assumendo che i job della matrice partano simultaneamente. Ciò introduce rumore nella stima del tempo di feedback.
- 4) **Ritardo di feedback stimato per massima verosimiglianza.** A differenza dei tempi di servizio di `failed` e `canceled`, il ritardo di revisione è stimato per massima verosimiglianza e non con il metodo dei momenti, per le ragioni discusse in §III-H. Ne consegue una sottostima di $E[D]$ di un fattore 3,4 rispetto alla media campionaria. La direzione dell'errore è nota e conservativa: la stima alternativa rafforzerebbe, anziché indebolire, la conclusione dello studio sul peso della revisione umana.
- 5) **Stazionarietà.** I parametri sono stimati su 15,7 mesi e trattati come costanti, mentre la configurazione della build matrix del progetto è verosimilmente cambiata nel periodo, come suggerisce l'assenza completa di build da 5 job nella distribuzione osservata.
- 6) **Fit accettati per adeguatezza pratica.** Con campioni dell'ordine di 10^4 il test di Kolmogorov–Smirnov è molto sensibile; il criterio adottato è la piccolezza di D unita all'aderenza grafica, non il non-rifiuto formale.

K. Collegamento con la fase di validazione

È bene fare una precisazione fin da subito: la validazione **non** confronta i parametri di configurazione con il dataset. Un confronto simile sarebbe circolare e per costruzione sempre superato, dato che i parametri sono stati estratti proprio da quel dataset.

La validazione confronta invece le grandezze **misurate dalle tracce prodotte dal simulatore** con le corrispondenti grandezze misurate sul dataset reale. Ciò che si verifica è dunque che il *motore di simulazione riproduca effettivamente*

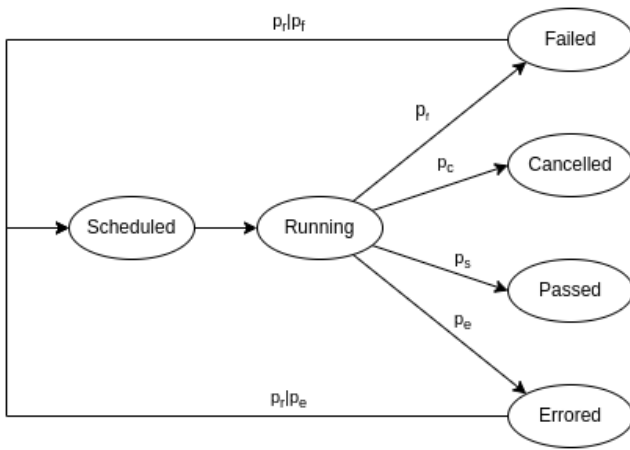


Figura 5. Modello degli stati dei job. Dagli stati *failed* ed *errored* si transita *sempre* allo stato di revisione umana; da questo si torna in coda con probabilità $p_{retry,failed}$ e $p_{retry,errored}$ rispettivamente, e si abbandona il sistema con probabilità complementare.

la statistica che gli è stata assegnata — proprietà tutt'altro che garantita a priori, data la complessità dell'instradamento, del loop di feedback e del modello bimodale degli *errored*. La fase di analisi qui descritta fornisce quindi tanto gli **input** del modello quanto il **riferimento oggettivo** rispetto a cui il modello viene giudicato.

IV. MODELLO CONCETTUALE

A. Ciclo di vita di un job

Una build genera uno o più job. Una volta entrati in esecuzione, i job sono sottoposti a due elaborazioni consecutive eseguite in successione dal medesimo runner:

- il *setup*, in cui viene predisposto l'ambiente di esecuzione secondo quanto specificato nella build matrix;
- il *test*, in cui vengono eseguiti i test sul codice nell'ambiente così predisposto.

Al termine dell'esecuzione, un job può trovarsi in quattro stati:

- *passed*: l'ambiente è stato predisposto correttamente e tutti i test sono andati a buon fine;
- *failed*: l'ambiente è stato predisposto correttamente, ma almeno un test è fallito;
- *errored*: si è verificato un errore a livello infrastrutturale durante la fase di setup o di test;
- *canceled*: l'utente ha forzatamente interrotto l'esecuzione del job; l'interruzione può avvenire in qualsiasi momento dell'esecuzione.

Il diagramma di transizione degli stati è riportato in Fig. 5.

B. Modellazione della rielaborazione

Poiché i job vengono eseguiti sul codice, è possibile che lo stesso codice produca più elaborazioni. Nel dataset sono infatti presenti job diversi lanciati a partire dallo stesso commit; in questi casi la mole di lavoro che i runner dovranno eseguire per

ogni esecuzione sarà identica. Dal punto di vista concettuale questo fenomeno è modellato in funzione dello stato del job.

Un job nello stato *passed* non verrà più rieseguito. Nonostante il dataset mostri diversi job rieseguiti anche se passati, questi vengono considerati come nuovi ingressi nel sistema: si tratta principalmente di esecuzioni periodiche automatiche (*cronjob*) piuttosto che di interventi umani. D'altra parte, un job può essere cancellato dal programmatore che si accorge di un errore nel codice dopo aver avviato la build e mentre questa è in esecuzione; di conseguenza il job in questione non sarà mai rieseguito, poiché il codice del progetto cambierà.

I job con stato *failed* o *errored* possono invece essere rieseguiti con una certa probabilità. Nello specifico, lo stato *errored* indica principalmente errori dovuti all'infrastruttura di Travis CI, come problemi di connettività, il che porta solitamente lo sviluppatore a rieseguire la build senza modifiche al codice. Lo stato *failed* richiede di norma modifiche al codice, ma non è detto che il programmatore se ne renda conto immediatamente: potrebbe dunque rieseguire la build con lo stesso codice. In generale, non si considera una rielaborazione certa in questi casi, dato che il codice sottostante potrebbe cambiare e, di conseguenza, anche il tempo di servizio dei job.

C. Struttura della rete di code

Il sistema si formalizza come una rete di code aperta a due nodi:

- 1) un **nodo di servizio** $M^{[B]}/G/c$, che rappresenta il pool di runner: arrivi poissoniani a gruppi di dimensione aleatoria B , tempo di servizio con distribuzione generale dipendente dall'esito, c serventi identici, coda unica a disciplina FIFO e capacità illimitata;
- 2) un **nodo di feedback** a infiniti serventi ($G/G/\infty$), che rappresenta la revisione umana: ogni job difettoso vi permane per un tempo lognormale indipendente dagli altri, senza contesa di risorse. La scelta di un nodo a infiniti serventi è la formalizzazione naturale dell'ipotesi che gli sviluppatori esaminino i propri fallimenti in parallelo e senza mutua interferenza.

L'arco di ritorno chiude la rete. Un job che completi il servizio con esito *failed* o *errored* transita **sempre** nel nodo di feedback: il ritardo di revisione è infatti sostenuto da ogni fallimento, anche da quelli che non daranno luogo a un riavvio, perché lo sviluppatore deve comunque esaminare l'esito prima di decidere. Solo al termine di tale ritardo il job rientra in coda al nodo di servizio, con probabilità p_{retry} dipendente dall'esito, oppure abbandona definitivamente il sistema con probabilità complementare. La distinzione è sostanziale ai fini del carico del nodo di feedback, che è attraversato da una frazione $p_{failed} + p_{errored} = 0,215$ dei job serviti e non dalla sola frazione $q = 0,167$ che effettivamente ritorna in coda.

V. MODELLO DELLE SPECIFICHE

Il sistema è modellato come un multiservente con c serventi, mostrato in Fig. 6. Ogni servente esegue in successione le due fasi descritte, secondo lo schema di Fig. 7.

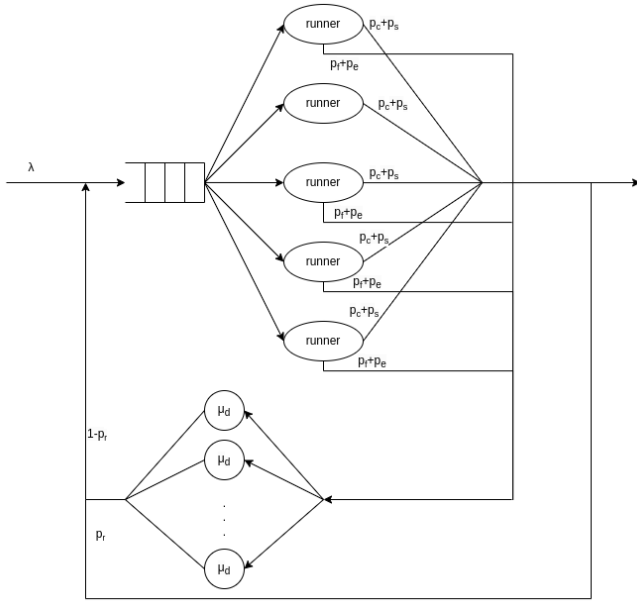


Figura 6. Modello analitico della rete. Il flusso esogeno λ alimenta la coda unica del pool di c runner; i job con esito *failed* o *errored* sono instradati al nodo di feedback a infiniti server e da lì, con probabilità p_{retry} , rientrano in coda.

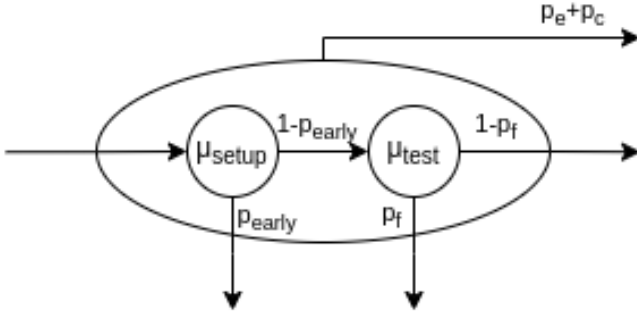


Figura 7. Modello del singolo runner. Il servizio si articola in due fasi consecutive; l'esito determina quale distribuzione governa la seconda fase e se questa venga eseguita per intero.

A. Processo di arrivo

Le nuove richieste in entrata seguono un processo di Poisson di parametro $\lambda = 4,6 \cdot 10^{-5} \text{ s}^{-1}$, corrispondente nella pratica a un nuovo arrivo ogni sei ore circa. Ad ogni arrivo il sistema riceve un gruppo di B job simultanei, con B distribuita secondo la distribuzione empirica della Tabella III.

B. Tempo di servizio

Il tempo di servizio di una singola visita al nodo è determinato dall'esito estratto per quel job, secondo lo schema seguente, dove S_{setup} segue la distribuzione empirica a 28 valori, $T \sim \text{Lognormale}(5,9545; 0,5316)$, $F \sim \text{Lognormale}(5,7644; 0,5958)$, $C \sim \text{Lognormale}(5,6105; 0,6168)$ e $U \sim \text{Uniforme}(0, 1)$:

$$S = \begin{cases} S_{\text{setup}} + T & \text{esito passed} \\ S_{\text{setup}} + F & \text{esito failed} \\ S_{\text{setup}} + C & \text{esito canceled} \\ U \cdot S_{\text{setup}} & \text{errored precoce } (p = 0,8664) \\ S_{\text{setup}} + T & \text{errored tardivo } (p = 0,1336) \end{cases} \quad (6)$$

I momenti risultanti del servizio misto sono $E[S] = 372,98 \text{ s}$ e $E[S^2] = 2,191 \cdot 10^5 \text{ s}^2$, da cui un coefficiente di variabilità $C_S^2 = 0,575$. Si osservi che la bimodalità degli errored innalza la variabilità del servizio rispetto a quella della sola componente lognormale ($C^2 = 0,327$): è la coesistenza di job da un secondo e job da sette minuti nella stessa coda a rendere il servizio più disperso.

L'entità dell'effetto va però quantificata nella metrica corretta. Nelle formule di attesa la dispersione del servizio entra attraverso il fattore $(1 + C_S^2)/2$ della Pollaczek-Khinchine, che passa da 0,664 a 0,788: la bimodalità inflaziona dunque la componente di attesa dipendente dal carico del **18,7%**, e non del 33% che il confronto fra i coefficienti di variazione (0,572 contro 0,758) lascerebbe supporre. È questa la ragione per cui la dispersione delle grandezze rilevanti per l'accodamento viene riportata come C^2 e non come CV : la seconda esagera sistematicamente l'impatto sulla coda, perché nelle formule di attesa il coefficiente di variazione non compare mai se non elevato al quadrato.

C. Nodo di feedback

Il ritardo dovuto all'intervento umano è modellato come un sistema a infiniti server in cui ogni server segue una distribuzione lognormale di parametri $\mu_d = 0,9591$ e $\sigma_d = 2,1399$, con tempi espressi in minuti. Vi accede *ogni* job che termini il servizio in stato *failed* o *errored*, indipendentemente dal fatto che venga poi rielaborato: il nodo rappresenta il tempo di esame dell'esito da parte dello sviluppatore, che precede e non presuppone la decisione di riavvio. Le probabilità che i job in stato *failed* ed *errored* vengano effettivamente rielaborati — ossia che, *al termine* del ritardo, rientrino in coda al nodo di servizio anziché abbandonare il sistema — valgono rispettivamente 0,3327 e 0,8637.

VI. MODELLO COMPUTAZIONALE

Il modello computazionale è implementato in Python secondo il paradigma *next-event*. Il codice sorgente è disponibile nella repository del progetto⁵.

A. Motore a eventi discreti

Il calendario degli eventi è realizzato con una coda di priorità a heap binario, che garantisce l'estrazione del minimo e l'inserimento in tempo logaritmico. Sono previsti tre tipi di evento: ARRIVAL, che genera un gruppo di job e schedula l'arrivo successivo; END_SERVICE, che libera un server e instrada il job verso l'uscita o verso il feedback; END_FEEDBACK, che decide il riavvio o l'abbandono. Ogni evento inserito nel

⁵<https://github.com/cantarelli-light-12pt/ProgettoPMCSN>

calendario porta con sé un contatore progressivo che funge da criterio di rottura dei pareggi, eliminando ogni ambiguità nell'ordinamento di eventi simultanei.

B. Generazione dei numeri pseudo-casuali

Come generatore è utilizzato il Lehmer multi-stream di Leemis e Park [1], con modulo $m = 2^{31} - 1$, moltiplicatore $a = 48271$ e 256 stream ottenuti per salto con moltiplicatore $A = 22925$. La scelta della coppia modulo-moltiplicatore segue le raccomandazioni di Park e Miller [7], che ne garantiscono periodo pieno e buone proprietà spettrali.

Ogni sorgente stocastica del modello utilizza uno **stream dedicato**: gli inter-arrivi, la dimensione del gruppo, l'esito, il setup, il test, il troncamento dell'errore precoce, il ritardo di feedback, la decisione di riavvio, il modo di errore e i servizi di *failed* e *canceled* impegnano undici stream distinti. Questa assegnazione persegue due scopi: garantire l'indipendenza fra le sorgenti ed evitare che una modifica della logica in un punto del modello alteri la sequenza di numeri consumata altrove. Quest'ultima proprietà si rivela decisiva nel confronto fra configurazioni diverse, come si discuterà in §XI-F.

I seed sono piantati una sola volta dal driver, non dal simulatore, così che repliche successive consumino gli stream in sequenza e risultino effettivamente indipendenti.

C. Stima delle metriche

Per il calcolo delle medie è utilizzato l'algoritmo di Welford [6] in due varianti: una versione standard per le metriche per-entità (tempo di servizio, di attesa e di risposta di ciascun job) e una versione pesata sul tempo per le metriche di stato (utilizzazione, popolazione in coda e nel sistema). Entrambe operano in linea con occupazione di memoria costante, senza necessità di conservare le osservazioni.

Gli intervalli di confidenza sono calcolati con la distribuzione t di Student sul campione delle medie di replica o di batch, secondo la formula $w = t_{1-\alpha/2, n-1} s / \sqrt{n}$, con deviazione standard campionaria (ddof = 1).

D. Tracce prodotte

Il simulatore produce quattro tracce distinte, mantenute in memoria durante la run e riversate su file al termine. Esse costituiscono l'unico ingresso delle fasi di verifica e validazione, che non hanno accesso allo stato interno del motore:

- la **traccia di stato**, con campioni dello stato del sistema (utilizzazione, lunghezza della coda, serventi occupati, job in feedback) su una griglia temporale fissa;
- la **traccia per job**, con una riga per ogni job che lascia definitivamente il sistema, contenente tempo di attesa, di servizio e di risposta complessivi;
- la **traccia di nodo**, con una riga per ogni completamento di servizio, contenente esito, attesa e servizio della singola visita. A differenza della precedente, essa copre *tutti* gli esiti, compresi i job che proseguono verso il feedback;

Tabella V
RIEPILOGO DEI CINQUE LIVELLI DI VERIFICA

	Livello	Controlli	Esito
1	Validità dei valori nelle tracce	11	11/11
2	Legge di Little sul nodo	3	3/3
3	Generatore pseudo-casuale	5	5/5
4	Generatori di variare	22	22/22
5	Caso degenerare $M^{[B]}/G/1$ e flusso	29	29/29
	Totale	70	70/70

- la **traccia degli arrivi**, con una riga per ogni evento di arrivo esogeno e la dimensione del gruppo generato.

Questa separazione fra traccia per job e traccia per visita è ciò che rende possibile verificare la legge di Little sul solo nodo di servizio, distinguendo il tempo di residenza nel nodo dal tempo di permanenza nel sistema, che include il feedback.

VII. VERIFICA

La verifica risponde alla domanda: *il programma implementa correttamente il modello concettuale?* È questione distinta dalla validazione, che riguarda invece l'aderenza del modello al sistema reale, e va condotta con strumenti propri: un simulatore può riprodurre accuratamente le statistiche del dataset e contenere al contempo errori di contabilità degli eventi che si manifestano solo in regimi diversi da quello nominale.

Si è pertanto organizzata la verifica su **cinque livelli**, dal più elementare al più strutturale, per un totale di **70 controlli automatici** eseguiti a ogni esecuzione della pipeline. Tutti i controlli operano sulle tracce descritte in §VI-D o su esecuzioni dedicate del motore; nessuno di essi utilizza il dataset reale. Il riepilogo è nella Tabella V.

A. Livello 1: validità dei valori raccolti

Il primo livello verifica che nessuna osservazione registrata violi i vincoli fisici e logici del modello. I controlli sono esaustivi, nel senso che vengono applicati a *ogni* riga di *ogni* traccia, e non a un campione.

Sulla run di regime a $c = 5$ essi hanno esaminato 330 856 record per job, 397 259 record per visita e 200 000 campioni di stato, verificando che:

- i tempi di attesa, di servizio e di risposta per job siano non negativi;
- valga per ogni job la disuguaglianza $R \geq W_q + S$, dove l'eventuale eccedenza corrisponde al tempo trascorso nel nodo di feedback;
- attesa e servizio di ciascuna visita al nodo siano non negativi;
- l'esito registrato appartenga all'insieme dei quattro esiti ammessi;
- l'utilizzazione sia compresa fra 0 e 1;
- il numero di serventi occupati sia compreso fra 0 e c ;
- la lunghezza della coda e il numero di job in feedback siano non negativi.

Il numero di violazioni riscontrate è **zero su tutti gli undici controlli**.

Tabella VI

VERIFICA DELLA LEGGE DI LITTLE SUL NODO ($c = 5$, ORIZZONTE 10^9 s)

Relazione	L (stato)	$X \cdot W$ (visite)	err. rel.
Nodo: $L = X(W_q + W_s)$	0,181660	0,181660	< 0,01%
Coda: $L_q = XW_q$	0,033587	0,033587	< 0,01%
Serventi: $L_s = XW_s$	0,148073	0,148073	< 0,01%

B. Livello 2: legge di Little sul nodo di servizio

Il secondo livello confronta due misure *indipendenti* della stessa grandezza. Da un lato L , il numero medio di job nel nodo, calcolato come media temporale esatta dal processo di stato mediante lo stimatore di Welford pesato sul tempo. Dall'altro $X \cdot W$, il prodotto fra il throughput del nodo, misurato come numero di visite diviso l'orizzonte, e il tempo medio di residenza, misurato dai tempi delle singole visite registrati nella traccia di nodo. Le due misure sono calcolate in maniera del tutto indipendente all'interno del codice. La loro coincidenza verifica che entrambe descrivano lo stesso fenomeno.

Il confronto è effettuato in forma scomposta, separando coda e serventi, così da localizzare un'eventuale discrepanza:

$$L_{\text{nodo}} = X(W_q + W_s) \quad (7)$$

$$L_q = XW_q \quad (8)$$

$$L_s = \rho c = XW_s \quad (9)$$

I risultati sono riportati nella Tabella VI: le due misure coincidono su tutte e tre le relazioni con un errore relativo inferiore a 10^{-4} , ampiamente entro la tolleranza del 5% fissata.

C. Livello 3: verifica del generatore pseudo-casuale

Poiché l'intero comportamento stocastico del modello poggia sul generatore di Lehmer, la sua correttezza è stata verificata esplicitamente con cinque controlli.

1) *Test di consistenza*: Il primo è il test di consistenza standard proposto dagli autori del generatore: partendo dallo stato $x_0 = 1$ ed eseguendo esattamente 10^4 estrazioni, lo stato deve valere $x_{10000} = 399\,268\,537$. Il valore ottenuto coincide esattamente.

2) *Disgiunzione degli stream*: Dopo l'inizializzazione con salto, i 256 stream devono partire da stati tutti distinti. Il controllo conta gli stati iniziali distinti e ne trova 256 su 256: nessuna coppia di stream parte dalla stessa posizione della sequenza.

3) *Uniformità marginale*: Per ciascuno degli undici stream effettivamente impiegati dal modello sono state generate $2 \cdot 10^5$ estrazioni e sottoposte a un test di Kolmogorov-Smirnov contro l'uniforme su $(0, 1)$. Il p -value minimo fra gli undici vale 0,3485, ben al di sopra della soglia del 5%.

4) *Incorrelazione*: Sono state calcolate le $\binom{11}{2} = 55$ correlazioni a coppie fra gli undici stream e le undici autocorrelazioni di ritardo unitario all'interno di ciascuno. Il criterio di esito è statistico: sotto l'ipotesi di indipendenza il coefficiente di correlazione campionario ha errore standard $1/\sqrt{n}$, per

Tabella VII

VERIFICA DEI GENERATORI DI VARIATE: MOMENTI EMPIRICI SU $2 \cdot 10^6$ CAMPIONI CONTRO MOMENTI TEORICI

Sorgente	$E[X]$ teor.	media emp.	err. rel.	z
Inter-arrivo, Exp	21739,13	21720,25	0,09%	1,23
Dimensione gruppo	7,1452	7,1472	0,03%	1,20
Tempo di setup	2,0242	2,0227	0,07%	1,04
Servizio test, Lognorm.	443,989	443,843	0,03%	0,81
Servizio failed	380,653	380,611	0,01%	0,24
Servizio canceled	330,538	330,724	0,06%	1,17
Ritardo feedback [min]	25,756	25,867	0,43%	0,62
Frazione passed	0,773436	0,773046	0,05%	1,32
Frazione failed	0,034525	0,034479	0,13%	0,36
Frazione errored	0,180305	0,180745	0,24%	1,62
Frazione canceled	0,011734	0,011731	0,03%	0,04
Riavvio dopo failed	0,332700	0,332461	0,07%	0,72
Riavvio dopo errored	0,863700	0,863746	0,01%	0,19
Errore precoce	0,866400	0,866665	0,03%	1,10

cui la statistica $z = |r|\sqrt{n}$ è approssimativamente normale standard. Tenuto conto della molteplicità dei $55 + 11 = 66$ confronti, si è adottata una soglia $z < 4$, corrispondente a un livello globale ben inferiore all'1%. Il valore massimo osservato è $|r| = 0,00526$ fra gli stream 3 e 5 ($z = 2,35$) per le correlazioni incrociate, e $|r_1| = 0,00592$ sullo stream 9 ($z = 2,65$) per le autocorrelazioni. Entrambi rientrano nella banda attesa.

D. Livello 4: verifica dei generatori di variate

Verificato il generatore uniforme, occorre verificare che le trasformazioni che da esso derivano le variato del modello producano effettivamente le distribuzioni richieste. Per ciascuna delle undici sorgenti stocastiche sono state generate $2 \cdot 10^6$ variato e sottoposte (ove possibile) a due controlli distinti: un confronto dei momenti e un test di adattamento.

1) *Confronto dei momenti*: Anche in questo caso il criterio di esito è statistico e non una soglia relativa fissa. La media campionaria di n variato indipendenti ha errore standard $\sigma/\sqrt{n}$, per cui si richiede

$$z = \frac{|\bar{x} - E[X]| \sqrt{n}}{\sigma} \leq 4. \quad (10)$$

Una soglia relativa fissa sarebbe stata metodologicamente inadeguata, perché le sorgenti del modello hanno coefficienti di variazione che differiscono di oltre un ordine di grandezza: dallo 0,33 della distribuzione dei gruppi al 9,8 del ritardo di feedback. Applicare a tutte la medesima tolleranza percentuale avrebbe significato essere eccessivamente permissivi con le prime ed eccessivamente severi con l'ultima.

I risultati sono nella Tabella VII. Tutte e quattordici le grandezze rientrano nella banda, con z massimo pari a 1,62.

Un dato merita commento: il ritardo di feedback presenta l'errore relativo maggiore (0,43%) pur avendo uno fra i valori più bassi di z . Non è una contraddizione, ma la manifestazione diretta del suo enorme coefficiente di variazione: con $CV = 9,8$ e $n = 2 \cdot 10^6$, l'errore standard relativo della media vale $CV/\sqrt{n} = 0,69\%$, cosicché uno scarto dello

0,43% è inferiore a un errore standard. Questo esempio illustra concretamente perché un criterio a soglia percentuale fissa sarebbe stato fuorviante, e anticipa un tema che tornerà in §XI-F: è questa sorgente a dominare la varianza delle stime di regime.

2) *Test di adattamento*: Alle sole verifiche sui momenti si è affiancato un test di adattamento completo sulla forma della distribuzione: chi-quadro per le distribuzioni discrete e per il meccanismo a rischi competitivi, Kolmogorov–Smirnov per l'esponenziale e per le quattro lognormali. Il test sui rischi competitivi replica esattamente la logica di ramificazione del simulatore e ne verifica la distribuzione a quattro categorie, controllando così non il generatore uniforme ma la correttezza delle soglie cumulate impiegate nel codice. Gli otto test sono tutti superati; i p -value sono compresi fra 0,17 e 0,92, ossia distribuiti nell'intervallo unitario come ci si attende sotto l'ipotesi nulla e senza addensamenti sospetti verso lo zero.

E. Livello 5: caso degenerare con soluzione analitica esatta

I livelli precedenti verificano i componenti; questo verifica il motore nel suo complesso, confrontandolo con una soluzione analitica *esatta* consentita dalla forzatura di alcuni parametri, senza modificare l'implementazione in sé. È il controllo più stringente dell'intera fase di verifica.

1) *Costruzione del caso degenerare*: Ponendo $p_{\text{passed}} = 1$ si disattiva integralmente l'anello di retroazione; ponendo $c = 1$ si riduce il pool a un singolo servente. Il modello degenera allora in una coda $M^{[B]}/G/1$: arrivi poissoniani a gruppi di dimensione B distribuita secondo la distribuzione empirica del progetto, servizio generale $S = S_{\text{setup}} + T$, un servente, disciplina FIFO. Per questa coda il tempo medio di attesa è noto in forma chiusa [3].

2) *Soluzione analitica*: Il tempo di attesa di un cliente si scompone in due contributi indipendenti.

Il primo è il lavoro non ancora smaltito che il gruppo cui il cliente appartiene trova nel sistema al proprio arrivo. Per la proprietà PASTA questo coincide con il carico di lavoro stazionario di una $M/G/1$ in cui ogni "cliente" è l'intero gruppo, con richiesta di servizio S_B pari alla somma di B servizi:

$$E[V] = \frac{\lambda E[S_B^2]}{2(1 - \rho)}, \quad E[S_B^2] = E[B]E[S^2] + E[B(B-1)]E[S]^2. \quad (11)$$

Il secondo è il servizio dei compagni di gruppo serviti prima di lui. Campionando un cliente a caso, la dimensione del suo gruppo è distorta dalla dimensione stessa, con $P(B^* = k) = k p_k / E[B]$, e la sua posizione è uniforme all'interno del gruppo; il numero atteso di clienti che lo precedono vale quindi $(E[B^2]/E[B] - 1)/2$.

Ne segue

$$E[W_q] = \frac{\lambda E[S_B^2]}{2(1 - \rho)} + \frac{E[S]}{2} \left(\frac{E[B^2]}{E[B]} - 1 \right), \quad (12)$$

con $\rho = \lambda E[B]E[S]$. Per $B \equiv 1$ la (12) degenera nella formula di Pollaczek–Khinchine classica [4], come deve.

Tabella VIII
CASO DEGENERARE $M^{[B]}/G/1$: TEMPO MEDIO DI ATTESA SIMULATO
CONTRO SOLUZIONE ANALITICA ESATTA, SU CINQUE LIVELLI DI
UTILIZZAZIONE

ρ	$E[W_q]$ teor. [s]	simulato [s]	err. rel.	IC copre
0,1	1 744,37	1 743,15 $\pm$ 6,30	0,07%	sì
0,3	2 327,10	2 321,34 $\pm$ 19,61	0,25%	sì
0,5	3 376,01	3 380,76 $\pm$ 50,19	0,14%	sì
0,7	5 823,47	5 771,67 $\pm$ 162,60	0,89%	sì
0,9	18 060,76	17 418,61 $\pm$ 1450,79	3,56%	sì

3) *Protocollo sperimentale*: Il confronto è stato ripetuto su **cinque livelli di utilizzazione**, $\rho \in \{0,1; 0,3; 0,5; 0,7; 0,9\}$, ottenuti variando λ a parità di tutti gli altri parametri secondo $\lambda = \rho / (E[B]E[S])$. Questa è una scelta deliberata: verificare il motore soltanto nel regime nominale ($\rho \approx 0,03$) lascerebbe non collaudata proprio la logica di accodamento, che nel regime nominale è raramente sollecitata. Ogni livello è simulato fino a servire circa 600 000 job, con eliminazione del 5% iniziale e stima per batch means su 64 batch.

Per ciascun livello sono confrontate cinque grandezze: utilizzazione, tempo medio di attesa, tempo medio di risposta, popolazione media in coda e popolazione media nel sistema, per un totale di 25 confronti.

4) *Risultati*: La Tabella VIII riporta il confronto sul tempo medio di attesa, che è la grandezza più sensibile e quella per cui la formula (12) è non banale.

L'accordo è eccellente. Sul tempo medio di attesa, in tutti e cinque i casi l'intervallo di confidenza al 95% costruito sulla simulazione **contiene il valore teorico**, e l'errore relativo resta sotto l'1% fino a $\rho = 0,7$. Complessivamente, 25 confronti su 25 sono superati, comprese le grandezze di stato: a titolo di esempio, a $\rho = 0,5$ la popolazione media in coda misurata vale $3,805 \pm 0,082$ contro un valore teorico di 3,785.

Va segnalato, per completezza, che dei 25 confronti uno solo non gode anche del ricoprimento dell'intervallo: l'utilizzazione a $\rho = 0,1$, misurata in $0,1028 \pm 0,0028$ contro un valore atteso di 0,1000, con un errore relativo del 2,8%. Il mancato ricoprimento è marginale — l'estremo inferiore dell'intervallo vale 0,10001 e manca il bersaglio per circa lo 0,4% della propria semi-ampiezza — ed è il livello in cui la stima *time-average* dell'utilizzazione è relativamente meno precisa, essendo il servente occupato solo per un decimo dell'orizzonte. Il controllo risulta comunque superato, perché il criterio di esito adottato è l'errore relativo entro il 5% e non il ricoprimento dell'intervallo, che sarebbe un criterio eccessivamente severo quando la grandezza confrontata è un parametro imposto per costruzione anziché una previsione del modello.

Il caso $\rho = 0,9$ merita una nota. L'errore relativo sale al 3,56% e la semi-ampiezza dell'intervallo cresce di un fattore 230 rispetto a $\rho = 0,1$. Non si tratta di un difetto del simulatore ma della fisica del problema: in prossimità della saturazione l'autocorrelazione dei tempi di attesa cresce drasticamente e i periodi di occupazione si allungano, cosicché 64 batch di 600 000 job complessivi non bastano più a decorrelare le medie di batch. La direzione dello scarto — la simulazione

sottostima leggermente — è quella tipica di un orizzonte insufficiente in regime quasi saturo, dove i rari periodi di congestione estrema che dominano la media non hanno avuto occasione di manifestarsi in numero sufficiente.

5) *Interpretazione:* Questo livello di verifica ha un valore particolare perché collauda simultaneamente e in modo non aggirabile: la generazione degli arrivi a gruppi con la corretta distribuzione; la logica di accodamento e di avvio del servizio; la contabilità dei tempi di attesa; il campionamento delle grandezze di stato; e la procedura dei batch means con la relativa stima intervallare. Un errore in uno qualsiasi di questi elementi si manifesterebbe come uno scostamento sistematico dalla curva teorica, e tale scostamento sarebbe crescente con ρ e quindi facilmente distinguibile dal rumore campionario. L'assenza di qualsiasi tendenza sistematica su cinque livelli che coprono un intervallo di utilizzazione di quasi un ordine di grandezza è l'evidenza più forte a favore della correttezza del motore.

F. Livello 5b: conservazione del flusso e fattore di visita

Il quinto livello si completa con quattro controlli di bilancio sulla configurazione nominale, che verificano relazioni strutturali della rete non riducibili a nessuno dei livelli precedenti.

1) *Conservazione dei job:* Il motore non può creare né distruggere job: la somma delle dimensioni dei gruppi generati deve eguagliare il numero di job usciti più quelli ancora presenti nel sistema al termine della run. Sulla run di regime si sono osservati 330 856 job generati e 330 856 job usciti, con **residuo esattamente nullo**. Il sistema si è cioè svuotato completamente entro l'orizzonte, il che è coerente con la bassissima utilizzazione, e nessun job è andato perduto né duplicato lungo il percorso attraverso coda, serventi e nodo di feedback.

2) *Fattore di visita:* Il numero medio di passaggi di un job dal nodo di servizio deve valere $v = 1/(1 - \rho)$, con ρ dato dalla (3). Il valore misurato, ottenuto come rapporto fra 397 259 visite e 330 856 job, vale 1,20070 contro il valore teorico 1,20079, con un errore relativo dello 0,01%. È la verifica diretta che l'anello di retroazione instrada i job con le probabilità corrette e che il ciclo termina come previsto.

3) *Throughput e utilizzazione:* Il throughput del nodo deve valere $X_{\text{nodo}} = \lambda E[B]v$: il valore misurato è $3,9726 \cdot 10^{-4}$ job/s contro $3,9468 \cdot 10^{-4}$ teorico, con uno scarto dello 0,65%. L'utilizzazione deve valere $\rho = X_{\text{nodo}} E[S]/c$ con $E[S]$ media pesata sui quattro esiti: il valore misurato è 0,029615 contro 0,029441 teorico, scarto dello 0,59%. Questo valore differisce lievemente dallo 0,029578 riportato nella Tabella XIV perché le due stime si riferiscono a porzioni diverse della traccia: il controllo di bilancio è calcolato sull'intera run, mentre la stima di regime è la media dei 64 batch successivi all'eliminazione del transitorio. Lo scarto fra le due, pari allo 0,13%, è ampiamente contenuto nella semi-ampiezza dell'intervallo di confidenza della seconda ($\pm 0,000678$, ossia il 2,3%).

Quest'ultimo controllo è più sottile di quanto appaia, perché richiede di calcolare il tempo medio di servizio tenendo conto

della struttura bimodale degli `errored`, il cui ramo precoce contribuisce $E[U] E[S_{\text{setup}}] = 1,012$ s e non zero. Il suo superamento verifica quindi che la (6) sia implementata esattamente come specificata, compreso il troncamento uniforme del setup.

G. Sintesi della verifica

Tutti i 70 controlli sono superati. La verifica è automatizzata ed è rieseguita a ogni esecuzione della pipeline, cosicché ogni modifica futura al motore verrebbe immediatamente confrontata con l'intero impianto. I report sono salvati in formato tabellare per consentirne l'ispezione indipendente.

Resta opportuno indicare *cosa la verifica non copre*. Essa non collauda il regime transitorio iniziale, perché tutti i confronti analitici riguardano grandezze di regime. Non collauda inoltre configurazioni con $c > 1$ contro una soluzione esatta, poiché per la $M^{[B]}/G/c$ non esiste una soluzione in forma chiusa; per queste ci si affida alla legge di Little e ai controlli di bilancio, che sono però condizioni necessarie e non sufficienti. Infine, la verifica assume corretta la libreria `scipy` impiegata per i quantili delle distribuzioni t , chi-quadro e per i test di adattamento, che è però software ampiamente collaudato e non fa parte del motore di simulazione.

VIII. VALIDAZIONE

La validazione risponde alla domanda: *il modello riproduce il comportamento del sistema reale?* Il modello computazionale è stato valutato rispetto ai dati del progetto `sonar-java` presenti nel dataset TravisTorrent.

A. Impostazione metodologica: evitare la circolarità

Il punto più importante dell'impostazione riguarda *che cosa* si confronta. Un confronto fra i parametri di configurazione e le statistiche del dataset sarebbe **circolare** e per costruzione sempre superato, dato che i parametri sono stati estratti proprio da quel dataset: si verificherebbe soltanto di non aver commesso errori di trascrizione.

Si è pertanto adottata una impostazione diversa. Tutti i valori del lato "simulato" sono **misurati dalle tracce** prodotte dal motore, ossia dai dati effettivamente realizzati dalla simulazione, e nessuno di essi è letto dai parametri di ingresso. Ciò che si verifica è dunque che il motore, messo in moto con quei parametri e fatto funzionare attraverso instradamento, accodamento e anello di retroazione, *riproduca all'uscita* la statistica che gli era stata assegnata all'ingresso. Non è proprietà garantita a priori: le probabilità di esito osservate all'uscita, per esempio, potrebbero differire da quelle di configurazione se il feedback loop reintrodusse i job in modo distorto, poiché i job che rientrano vengono riesaminati e generano nuove osservazioni di esito.

Le grandezze confrontate sono cinque: distribuzione degli esiti, probabilità di feedback, tempo di servizio condizionato all'esito, distribuzione della dimensione dei gruppi e tasso di arrivo. Il confronto è condotto sulla run di regime a $c = 5$ (orizzonte 10^9 s, 330 856 job usciti e 397 259 visite al nodo), che fornisce un campione simulato di poco più di un ordine di grandezza più numeroso del riferimento reale.

Tabella IX

VALIDAZIONE DELLA DISTRIBUZIONE DEGLI ESITI E DELLA PROBABILITÀ DI FEEDBACK

Grandezza	Reale	Simulato	diff
Esito passed	0,7734	0,7734	< 0,0001
Esito failed	0,0345	0,0347	0,0002
Esito errored	0,1803	0,1802	0,0001
Esito canceled	0,0117	0,0116	0,0001
Prob. di feedback (failed+errored)	0,2148	0,2149	0,0001

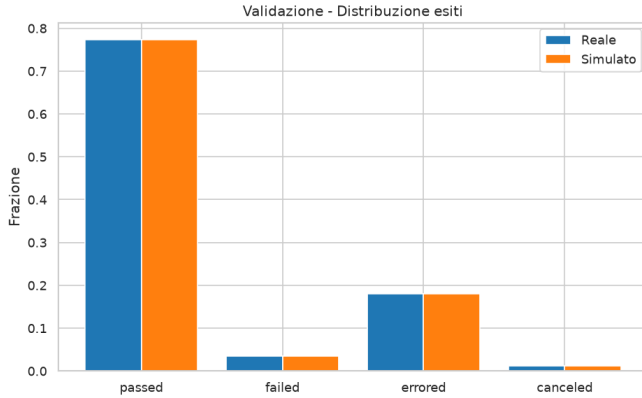


Figura 8. Confronto fra le probabilità che un job termini in ciascuno dei quattro stati, misurate sul dataset reale e sulle tracce della simulazione.

B. Distribuzione degli esiti e probabilità di feedback

Le frazioni di job che terminano in ciascuno dei quattro stati sono misurate dalla traccia di nodo, che registra una riga per ogni completamento di servizio e copre quindi anche le visite dei job che proseguono verso il feedback. Il confronto è riportato nella Tabella IX.

Lo scarto massimo è di $2 \cdot 10^{-4}$ in valore assoluto, ossia dello 0,6% in valore relativo. Per collocare correttamente questo risultato occorre osservare che l'errore standard della frazione reale, stimata su 16278 job, vale $\sqrt{p(1-p)/n} \approx 8,4 \cdot 10^{-4}$ per l'esito canceled: gli scarti osservati sono dunque inferiori all'incertezza del riferimento stesso, e non sarebbe metodologicamente corretto pretendere di più.

La probabilità di feedback è la grandezza che governa l'intensità dell'anello di retroazione e dunque il fattore di visita; il suo scarto di 10^{-4} conferma che il volume di traffico endogeno generato dal modello è quello atteso.

C. Tempo di servizio condizionato all'esito

Il confronto sui tempi di servizio è condotto separatamente per ciascun esito, poiché il modello tratta i quattro esiti con distribuzioni distinte (§III-G); un confronto aggregato maschererebbe eventuali compensazioni fra sovrastime e sottostime. Il riferimento reale è la durata di build registrata nel dataset; per gli errored, coerentemente con il modello bimodale, i job privi di durata registrata sono trattati come servizio nullo.

I risultati sono riportati nella Tabella X.

Tabella X

VALIDAZIONE DEI TEMPI MEDI DI SERVIZIO PER ESITO

Esito	n_R	reale [s]	n_S	sim. [s]	diff. [s]
passed	9731	439,50	307249	445,72	+6,22
failed	562	382,17	13785	379,95	-2,22
errored	2935	57,72	71601	60,59	+2,87
canceled	71	327,28	4624	335,25	+7,96

Tabella XI

SCOMPOSIZIONE DEL RESIDUO DI +6,51 S SUL TEMPO DI SERVIZIO DEI JOB PASSED

Causa	Contributo
(a) Media implicata dalla lognormale a massima verosimiglianza (443,989 s) superiore alla media empirica della durata netta di test (440,078 s)	+3,911 s
(b) Media della distribuzione empirica del setup, stimata su tutti i job (2,024 s), superiore al setup medio dei soli passed (1,622 s)	+0,402 s
(c) Differenza fra il sottoinsieme di calibrazione ($n = 9078$, richiede setup e durata entrambi positivi, media 441,700 s) e il sottoinsieme di validazione ($n = 9731$, richiede la sola durata, media 439,502 s)	+2,198 s
Totale	+6,511 s

Gli scarti sono tutti inferiori all'1,5% in valore relativo. Limitarsi a constatarlo sarebbe però insufficiente: uno scarto piccolo ma sistematico può nascondere un errore di modellazione, e la sua interpretazione richiede che se ne identifichi la causa. Si procede dunque a scomporre analiticamente ciascun residuo.

1) *Esito passed: scomposizione esatta del residuo:* Il modello genera un servizio atteso pari a $E[S_{\text{setup}}] + E[T] = 2,024 + 443,989 = 446,013$ s, contro un riferimento reale di 439,502 s: un residuo teorico di +6,511 s, di cui la simulazione ne realizza +6,22. Tale residuo si scompone *esattamente* in tre contributi, riportati nella Tabella XI.

Le tre cause sono di natura diversa e vanno giudicate diversamente.

Il contributo (a) è il più interessante ed è un **effetto sistematico del metodo di stima**: la massima verosimiglianza per la lognormale ottimizza l'aderenza in scala logaritmica e, di fronte a un campione con coda destra più pesante di una lognormale, produce parametri la cui media aritmetica implicata eccede la media empirica. È esattamente lo stesso fenomeno che, in forma molto più marcata (fino all'80%), aveva imposto di ricorrere al metodo dei momenti per i failed e i canceled; qui l'effetto è contenuto allo 0,9% perché il campione dei passed è assai meno disperso. In retrospettiva, l'adozione del metodo dei momenti anche per la fase di test avrebbe annullato questo contributo.

Il contributo (b) discende da una scelta deliberata: la distribuzione del setup è stimata su *tutti* i job e non sui soli passed, perché nel modello la fase di setup precede la determinazione dell'esito ed è quindi comune a tutti i rami. Il

residuo di 0,4 s è il prezzo, trascurabile, di questa coerenza modellistica.

Il contributo (c) non è un errore del modello ma un artefatto del confronto: calibrazione e validazione operano su sottoinsiemi leggermente diversi dei `passed`, perché la prima richiede che siano registrati sia il setup sia la durata (ed entrambi positivi), mentre la seconda richiede la sola durata. I 653 job in più del sottoinsieme di validazione hanno durata mediamente inferiore.

Il fatto che i tre contributi sommino esattamente al residuo osservato ($3,911 + 0,402 + 2,198 = 6,511$) conferma che non vi sono cause residue non identificate, e in particolare che non vi è alcun errore nella logica di composizione delle due fasi di servizio.

2) *Esito errored: verifica della bimodalità:* Anche il residuo degli `errored` si scompone. Il valore atteso dal modello vale

$$E[S_{\text{err}}] = p_{\text{early}} E[U]E[S_{\text{setup}}] + (1 - p_{\text{early}})(E[S_{\text{setup}}] + E[T]) \\ = 0,8664 \cdot 1,012 + 0,1336 \cdot 446,013 = 60,46 \text{ s, (13)}$$

in ottimo accordo con i 60,59 s misurati dalla simulazione. Il riferimento reale vale invece $0,8664 \cdot 0 + 0,1336 \cdot 432,14 = 57,73$ s, in accordo esatto con i 57,72 s misurati sul dataset. Il residuo *teorico* vale dunque +2,73 s, di cui la simulazione ne realizza +2,87; i 0,14 s di differenza sono fluttuazione campionaria delle due misure. Il residuo teorico si ripartisce esattamente in due contributi: +0,88 s dovuti al fatto che il modello assegna agli errori precoci una frazione del setup anziché zero ($0,8664 \cdot 1,012$), e +1,86 s dovuti al fatto che gli errori tardivi riutilizzano la distribuzione dei `passed` (media 446,0 s) invece della media effettivamente osservata per essi, 432,1 s ($0,1336 \cdot 13,87$). La somma, +2,730 s, coincide con il residuo teorico alla terza cifra decimale.

Questa scomposizione è significativa perché conferma che il modello bimodale è implementato correttamente e che la sua approssimazione residua è nota e quantificata. Si osservi che il valore aggregato di 57,72 s è privo di significato fisico di per sé: nessun job `errored` dura realmente 57 secondi. È la media di due popolazioni distinte, l'una a circa 0 s e l'altra a circa 432 s. Il confronto sulla sola media sarebbe quindi debole, ed è per questo che si affianca ad esso il confronto sull'intera funzione di ripartizione empirica (Fig. 9), dove il gradino all'86,6% in corrispondenza dell'origine rende visivamente evidente la corretta riproduzione della bimodalità.

3) *Esiti failed e canceled: residui e limite del riferimento:* Per i `failed` il valore atteso dal modello vale $2,024 + 380,653 = 382,677$ s contro un riferimento reale di $1,490 + 380,682 = 382,172$ s: un residuo teorico di appena +0,505 s, che si scompone in modo esatto nei contributi già introdotti. Il contributo (b) è dominante e vale +0,534 s: è lo stesso già isolato per i `passed`, ossia il setup medio stimato su tutti i job (2,024 s) contro il setup medio dei soli `failed` (1,490 s). Il contributo (a) vale -0,029 s ed è imputabile al solo arrotondamento di μ e σ alla quarta cifra decimale, che porta la media implicata dalla lognormale a 380,653 s contro i 380,682 s della media empirica netta. Il

contributo di sottoinsieme (c) è invece **nullo**, perché qui calibrazione e validazione insistono sugli stessi 562 job. La somma $-0,029 + 0,534 = +0,505$ s esaurisce esattamente il residuo. Azzerato (c), esso si riduce di fatto al solo disallineamento del setup, poiché l'errore introdotto dalla stima non arriva a tre centesimi di secondo: è la conferma più netta dell'efficacia del metodo dei momenti presente nel documento. Lo scarto effettivamente misurato è di -2,22 s, differenza attribuibile alla fluttuazione campionaria.

Per i `canceled` il modello attende $2,024 + 330,538 = 332,56$ s contro un riferimento reale di 327,28 s: un residuo teorico di +5,28 s, che si scompone anch'esso in modo esatto nei tre contributi già introdotti. Il contributo (a) vale -0,01 s ed è imputabile al solo arrotondamento di μ e σ alla quarta cifra decimale: anche qui il metodo dei momenti è esatto. Il contributo (b) vale +0,63 s, differenza fra il setup medio su tutti i job (2,024 s) e quello dei soli `canceled` (1,394 s). Il contributo (c) vale +4,66 s ed è il più istruttivo: il sottoinsieme di calibrazione ($n = 70$, media 330,550 s) esclude l'unico job la cui durata netta non è positiva, mentre il riferimento di validazione ($n = 71$) lo include, con una media delle durate nette di 325,891 s. Quel job presenta una `tr_log_builddduration` inferiore al proprio `tr_log_setup_time`, ossia una durata netta di -0,300 s: si tratta di un'incoerenza del dato grezzo, non di una build molto breve. I tre contributi sommano a +5,28 s ed esauriscono il residuo teorico. Ne segue che **l'88% del residuo dei canceled è prodotto da un singolo record incoerente del dataset**, non da un'approssimazione del modello.

Resta da giudicare lo scarto effettivamente *misurato*, pari a +7,96 s: sottratto il residuo teorico, i restanti +2,68 s sono rumore campionario. Ed è a questo proposito che il caso dei `canceled` illustra un limite intrinseco della validazione, che merita di essere esplicitato. Il riferimento reale è costituito da soli **71 job**, con deviazione standard campionaria di 226,7 s: l'errore standard della media reale vale dunque $226,7/\sqrt{71} = 26,9$ s. Lo scarto complessivo di 7,96 s è pari a 0,30 errori standard, ossia perfettamente compatibile con l'ipotesi che i due valori coincidano. In altri termini, **per questo esito il fattore limitante della validazione non è la precisione del simulatore — che dispone di 4624 osservazioni — ma la scarsità del dato reale**. Nessun raffinamento del modello potrebbe essere distinto dal rumore con un riferimento così esiguo, e sarebbe scorretto interpretare uno scarto entro tale banda come evidenza di disaccordo o di accordo.

Analogo ragionamento, in forma meno severa, vale per i `failed`, il cui errore standard di riferimento è di 11,20 s: anche in questo caso lo scarto osservato è inferiore a un quinto di errore standard.

D. Distribuzione della dimensione dei gruppi

La distribuzione dei gruppi è misurata dalla traccia degli arrivi, contando le dimensioni effettivamente generate dal motore lungo tutta la run. Il confronto con la distribuzione empirica reale è riportato nella Tabella XII.

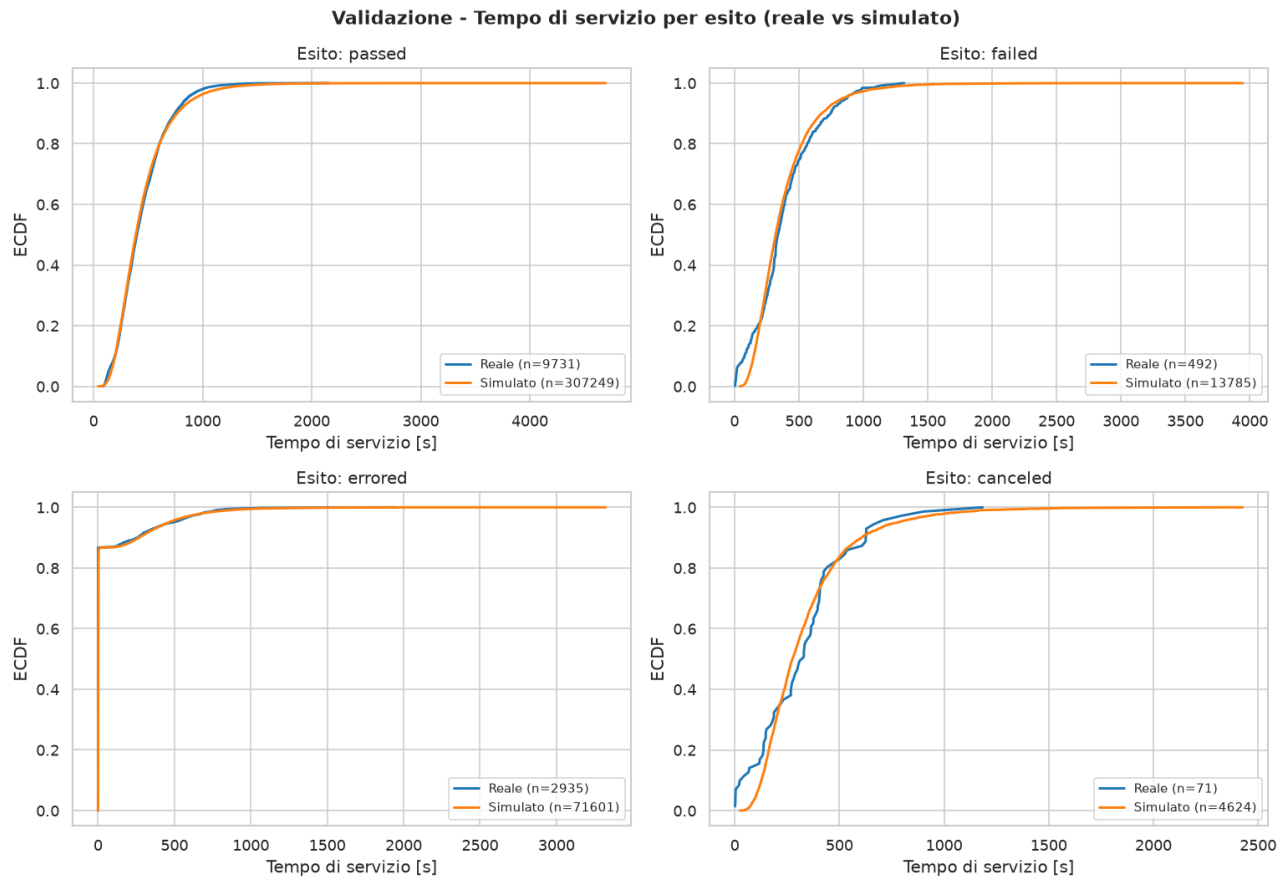


Figura 9. Confronto fra le distribuzioni dei tempi di servizio reali e simulati per ciascuno dei quattro esiti. Si osservi in particolare il pannello *errored*, dove il gradino iniziale all'86,6% riproduce la popolazione a rottura precoce, e il pannello *canceled*, dove la maggiore irregolarità della curva reale riflette la sua esigua numerosità ($n = 71$).

Tabella XII
VALIDAZIONE DELLA DISTRIBUZIONE DELLA DIMENSIONE DEI GRUPPI

Job/build	1	2	3	4	6	7	8	9	10
Reale	,0426	,0189	,0457	,0755	,1058	,0781	,3468	,1962	,0904
Simulato	,0426	,0190	,0459	,0767	,1059	,0784	,3474	,1939	,0903
diff	,0001	,0001	,0003	,0012	,0001	,0002	,0006	,0024	,0001

Lo scarto massimo vale $2,4 \cdot 10^{-3}$, in corrispondenza del valore 9. Va sottolineato che il modello **non** genera mai gruppi di dimensione 5, riproducendo così il buco strutturale osservato nel dataset: la scelta di una distribuzione empirica anziché parametrica (§III) trova qui la sua conferma diretta. Una qualunque famiglia discreta standard avrebbe assegnato massa positiva al valore 5, producendo uno scarto su quel punto.

E. Tasso di arrivo

Il tasso di arrivo dei nuovi gruppi è misurato come rapporto fra il numero di eventi di arrivo generati e l'orizzonte simulato. Il riferimento reale è ottenuto con lo stesso rapporto sulle sole build esogene — le 1903 selezionate dal criterio di §III-C

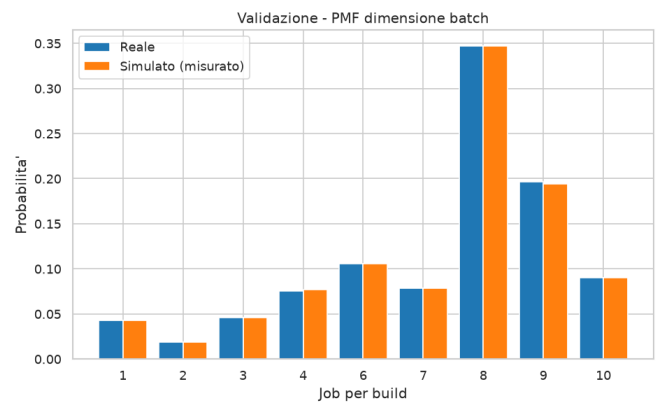


Figura 10. Confronto fra la distribuzione reale del numero di job per build e quella realizzata dal simulatore. L'assenza della barra in corrispondenza del valore 5 è riprodotta esattamente.

— e non tramite un filtro sostitutivo: la validazione invoca la medesima procedura usata in calibrazione, così che le due grandezze confrontate poggino per costruzione sulla stessa popolazione. Si ottengono $4,6193 \cdot 10^{-5}$ build/s per il dato

reale contro $4,6366 \cdot 10^{-5}$ build/s misurati su 46 366 arrivi simulati, con un errore relativo dello **0,38%**.

Anche in questo caso vale la pena valutare l'ordine di grandezza atteso dell'errore: il riferimento reale è stimato su 1 903 arrivi, con un errore standard relativo di $1/\sqrt{1903} = 2,3\%$. Lo scarto osservato è dunque pari a circa un sesto di errore standard del riferimento.

a) *Due stimatori dello stesso tasso*: Il valore reale appena riportato non coincide con l'inverso del tempo medio di inter-arrivo di §III-D ($21\,820$ s, ossia $4,5829 \cdot 10^{-5}$ build/s): i due stimatori distano lo 0,79%. La ragione è che il fit esponenziale scarta i quattordici inter-arrivi nulli — build esogene distinte avviate entro lo stesso secondo — perché una esponenziale con origine bloccata in zero non li ammette, mentre il rapporto fra numero di arrivi e ampiezza della finestra osservata li conta a pieno titolo. Qui si adotta il secondo, perché è l'analogo esatto della grandezza misurata lato simulatore, che è appunto un conteggio diviso un orizzonte. La distinzione non tocca il modello: entrambi gli stimatori arrotondano al $\lambda = 4,6 \cdot 10^{-5}$ adottato.

b) *Potere validante del confronto*: È onesto riconoscere che questo è il più debole dei cinque confronti. Il tasso di arrivo non è una previsione del modello ma un suo parametro di ingresso, calibrato su questi stessi dati: il valore “simulato” è la realizzazione campionaria di λ sull'orizzonte considerato, e il suo accordo con il riferimento è in larga misura garantito per costruzione. Il contenuto effettivo del controllo è dunque di natura diversa da quello degli altri quattro — verifica che la pipeline di calibrazione e il generatore di arrivi a gruppi siano fra loro coerenti, ossia che il processo di Poisson composto realizzi il tasso inteso e che il riferimento sia estratto dalla popolazione giusta. Non è per questo un controllo ozioso: è l'unico punto in cui il criterio di separazione fra traffico esogeno ed endogeno, la scelta metodologicamente più delicata dell'intera calibrazione, viene messo alla prova da un capo all'altro, dal dataset fino agli arrivi effettivamente generati dal motore. Il limite resta quello indicato al punto 3 di §VIII-F: due processi con lo stesso tasso medio ma diversa struttura di correlazione superano entrambi il confronto.

F. Sintesi e limiti della validazione

Il modello riproduce tutte e cinque le grandezze di riferimento entro l'incertezza statistica del riferimento stesso. Laddove si osservano scarti sistematici — il caso dei tempi di servizio — essi sono stati scomposti analiticamente nelle loro cause, tutte identificate e quantificate, e nessuna di esse rivela un errore di implementazione.

Restano tuttavia da esplicitare tre limiti della validazione condotta.

- 1) **Validazione sulle distribuzioni marginali, non sulla dinamica**. Il confronto riguarda le distribuzioni marginali delle grandezze in ingresso e in uscita dal servizio, non la dinamica temporale del sistema. Le grandezze prestazionali di maggiore interesse — tempo di risposta, tempo di attesa, lunghezza della coda — **non sono validabili**, perché il dataset non registra né l'istante

di accodamento né il numero di runner effettivamente disponibili in ciascun momento. Su di esse ci si affida integralmente alla verifica (§VII), e in particolare al confronto con la soluzione analitica esatta del caso degenero.

- 2) **Precisione limitata dal riferimento**. Come mostrato per i canceled e per il tasso di arrivo, per più di una grandezza l'errore standard del dato reale eccede lo scarto osservato. La validazione non può quindi discriminare fra il modello attuale e un modello leggermente diverso: essa stabilisce la compatibilità, non l'unicità.
- 3) **Ipotesi non validate**. L'ipotesi di arrivi poissoniani è stata adottata pur essendo esplicitamente rifiutata dai dati (§III-J), e la validazione sul solo tasso medio di arrivo non la mette alla prova: due processi con lo stesso tasso medio ma diversa struttura di correlazione superano entrambi questo confronto. Analogamente, l'ipotesi di indipendenza fra i tempi di servizio dei job appartenenti alla stessa build matrix non è verificabile con i dati disponibili, benché sia plausibilmente violata: job della stessa build eseguono lo stesso codice, e le loro durate sono verosimilmente correlate positivamente. Tale correlazione, se presente, aumenterebbe la variabilità del carico di lavoro portato da un gruppo e quindi i tempi di attesa reali rispetto a quelli simulati.

IX. ANALISI DEL TRANSITORIO

Per ciascuna delle quattro configurazioni di serventi è stata condotta un'analisi del transitorio con il **metodo delle repliche indipendenti** su orizzonte finito. Si eseguono cinque run con cinque semi distinti (12345, 23456, 34567, 45678, 56789), ciascuna di durata 10^7 s (circa 116 giorni simulati), e per ognuna si traccia la *media cumulata* progressiva delle grandezze di interesse. La sovrapposizione delle cinque curve sullo stesso grafico rende visibili simultaneamente la convergenza al regime e la variabilità fra le repliche.

L'analisi persegue due scopi: individuare la durata del transitorio iniziale, per dimensionare correttamente il periodo di riscaldamento da eliminare nell'analisi di regime, e fornire una stima a orizzonte finito confrontabile con quella a orizzonte infinito.

A. Convergenza delle diverse metriche

Le tre grandezze osservate mostrano comportamenti di convergenza nettamente diversi, e la differenza è informativa.

L'**utilizzo** (Fig. 11) presenta un picco iniziale molto pronunciato — fino a valori prossimi all'unità per $c = 1$ — dovuto al fatto che il sistema parte vuoto e il primo gruppo di job, di dimensione media 7,1, satura istantaneamente il pool. Le curve si assestano rapidamente e sono sostanzialmente stabili già dopo circa $2 \cdot 10^6$ s, ossia dopo circa il 20% dell'orizzonte finito.

Il **tempo di servizio dei job passed** (Fig. 12) converge ancora più rapidamente, stabilizzandosi attorno a 446 s entro le prime poche centinaia di job. Il motivo è strutturale: il tempo di servizio non dipende dallo stato del sistema, cosicché

Tabella XIII
STIME A ORIZZONTE FINITO (10^7 s, 5 REPLICHE INDIPENDENTI): MEDIA
± SEMI-AMPIEZZA DELL'INTERVALLO DI CONFIDENZA AL 95%

<i>c</i>	Utilizzazione	Servizio <i>passed</i>	Tempo di risposta
1	0,1419 ± 0,0117	446,88 ± 2,08	2702,15 ± 158,10
2	0,0706 ± 0,0068	446,86 ± 2,14	1562,80 ± 172,01
5	0,0299 ± 0,0066	446,82 ± 2,11	1006,84 ± 179,16
10	0,0153 ± 0,0039	446,91 ± 1,97	908,05 ± 179,15

la sua media cumulata è semplicemente la media di variabili indipendenti e identicamente distribuite e non presenta alcun transitorio in senso proprio. La sua inclusione è nondimeno utile come controllo: l'assenza di deriva conferma che il motore non introduce dipendenze spurie fra servizio e stato.

Il **tempo di risposta** (Fig. 13) è la grandezza che converge più lentamente e con la dispersione maggiore fra repliche. Ciò è atteso e ha una causa precisa, individuata in §XI-F: il tempo di risposta incorpora il ritardo di feedback umano, la cui distribuzione lognormale con $\sigma = 2,14$ ha coefficiente di variazione pari a 9,8. Ogni singolo evento di feedback eccezionalmente lungo produce un salto visibile nella media cumulata, e occorrono molte osservazioni perché la media si stabilizzi.

B. Risultati numerici e scelta del riscaldamento

La Tabella XIII riporta, per ciascuna configurazione, la media sulle cinque repliche della media cumulata finale, con l'intervallo di confidenza al 95% calcolato con la *t* di Student a 4 gradi di libertà.

Due osservazioni si impongono. La prima riguarda la **precisione relativa** molto diversa fra le tre grandezze: il tempo di servizio è stimato con un'incertezza relativa dello 0,5%, l'utilizzazione con un'incertezza compresa fra l'8% e il 26%, il tempo di risposta con un'incertezza fra il 6% e il 20%. Con sole cinque repliche e un orizzonte di 10^7 s — che a $c = 5$ produce circa 3 100 job usciti per replica — la precisione ottenibile su una grandezza così dispersa è strutturalmente limitata. È questa la ragione principale per cui l'analisi di regime è stata condotta con i batch means su una singola run lunga, che a parità di costo computazionale fornisce un numero di osservazioni di poco più di un ordine di grandezza superiore.

La seconda osservazione riguarda il **confronto con le stime di regime** (Tabella XIV). Le stime a orizzonte finito risultano sistematicamente superiori: per il tempo di risposta a $c = 5$, 1 006,84 s contro 972,93 s, e per l'utilizzazione 0,0299 contro 0,0296. Il segno della discrepanza è quello atteso, poiché l'orizzonte finito include il picco transitorio iniziale nella media cumulata, e la sua entità è compatibile con gli intervalli di confidenza.

Sulla base dell'analisi del transitorio si è fissato il periodo di riscaldamento dell'analisi di regime a $5 \cdot 10^6$ s, valore superiore di un fattore 2,5 al tempo di assestamento osservato per l'utilizzazione e pari allo 0,5% dell'orizzonte della run

lunga: una scelta deliberatamente conservativa, il cui costo in termini di osservazioni scartate è trascurabile.

X. ANALISI A ORIZZONTE INFINITO

L'analisi di regime è condotta con il metodo dei **batch means** [5]. Si esegue una singola run di durata 10^9 s (circa 31,7 anni simulati), si elimina il transitorio iniziale di $5 \cdot 10^6$ s e si divide il resto in $k = 64$ batch di uguale durata. Per ciascun batch si calcola la media della grandezza di interesse; i 64 valori così ottenuti forniscono la stima puntuale e l'intervallo di confidenza, calcolato con la *t* di Student a 63 gradi di libertà. Le grandezze mediate sul tempo (utilizzazione, popolazioni) sono calcolate sui campioni di stato del batch, quelle per job sui job usciti nel batch.

La run produce 330 856 job usciti e 397 259 visite al nodo, corrispondenti a circa 5 142 job per batch: numerosità ampiamente sufficiente a giustificare l'ipotesi di approssimativa indipendenza fra medie di batch, sulla quale poggia la stima intervallare.

A. Diagnostica dei batch

Prima di utilizzare le stime occorre verificare che i batch siano sufficientemente lunghi da risultare approssimativamente incorrelati. A tal fine si riportano, per ciascuna grandezza, i valori dei 64 batch con sovrapposte la media e la banda di confidenza (Figg. 14–17). L'ispezione dei diagrammi non evidenzia né derivate sistematiche né oscillazioni a bassa frequenza, che sarebbero il sintomo di batch troppo corti.

Fa parziale eccezione il tempo di risposta (Fig. 15), dove si osservano alcuni batch nettamente fuori scala, in corrispondenza degli indici 33–36. Non si tratta di un difetto della procedura ma della manifestazione diretta della coda pesante del ritardo di feedback: un singolo job che permanga in revisione per settimane sposta visibilmente la media del batch in cui esce. Il fenomeno è presente in modo identico in tutte e quattro le configurazioni, il che ne conferma l'origine — è lo stesso evento, riprodotto dagli stessi numeri pseudo-casuali (§XI-F) — e giustifica la maggiore ampiezza relativa dell'intervallo di confidenza su questa grandezza.

B. Risultati di regime

La Tabella XIV riporta le stime di regime per tutte e quattro le configurazioni.

XI. DISCUSSIONE DEI RISULTATI

A priori, secondo l'intuito, sarebbe possibile affermare che l'aumento del numero di serventi porti a prestazioni migliori. Tuttavia, i risultati della Tabella XIV consentono una lettura assai più informativa. Questa sezione ne articola l'interpretazione in sei punti, ciascuno dei quali conduce a una conclusione di rilevanza pratica.

Transitorio - Utilizzazione (media cumulata)

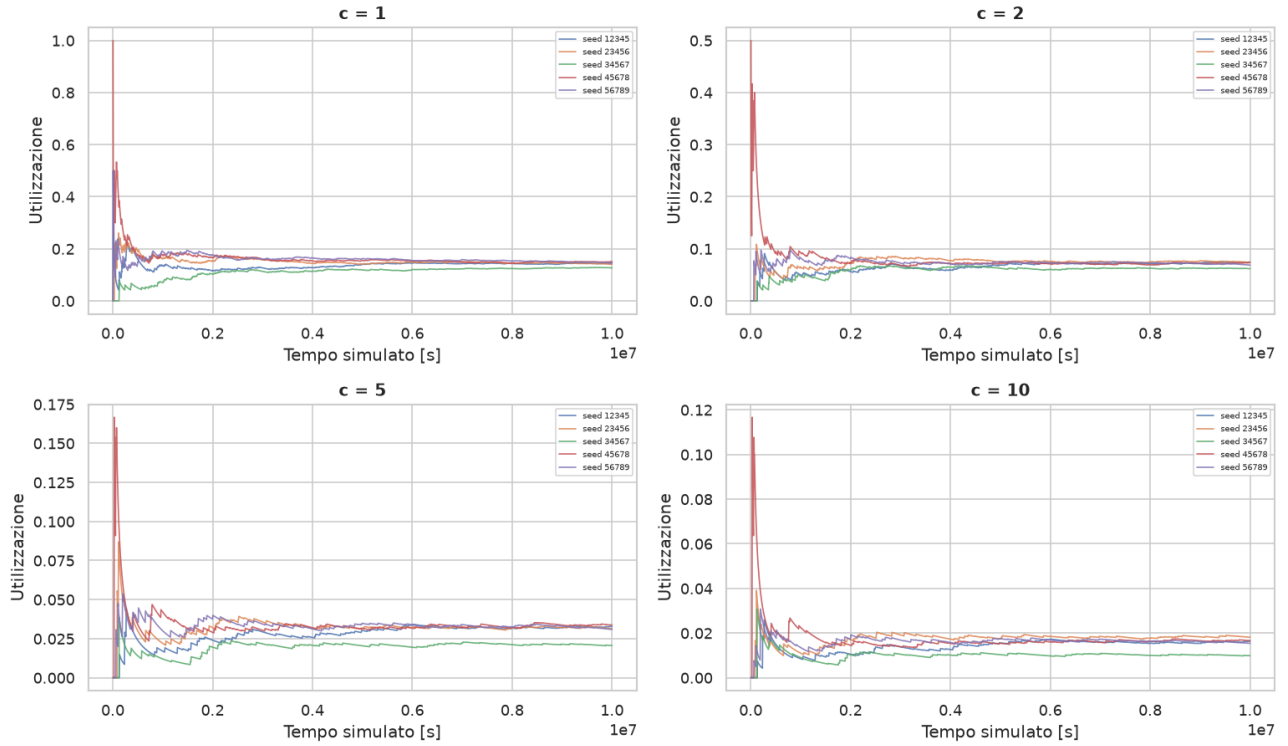


Figura 11. Analisi del transitorio dell'utilizzazione dei server al variare del loro numero. Ogni pannello riporta le cinque repliche indipendenti; l'ascissa è il tempo simulato.

Tabella XIV

STIME DI REGIME AL VARIARE DEL NUMERO DI SERVERI (BATCH MEANS, 64 BATCH, MEDIA $\pm$ SEMI-AMPIEZZA DELL'INTERVALLO DI CONFIDENZA AL 95%)

Grandezza	$c = 1$	$c = 2$	$c = 5$	$c = 10$
Utilizzazione	$0,147723 \pm 0,001830$	$0,073271 \pm 0,001244$	$0,029578 \pm 0,000678$	$0,014734 \pm 0,000364$
Tempo medio di attesa [s]	$1791,17 \pm 14,13$	$658,05 \pm 4,71$	$101,45 \pm 0,98$	$1,47 \pm 0,17$
Tempo medio di risposta [s]	$2662,83 \pm 45,66$	$1529,75 \pm 44,36$	$972,93 \pm 44,17$	$873,05 \pm 44,37$
Tempo di servizio <i>passed</i> [s]	$445,68 \pm 1,07$	$445,69 \pm 1,07$	$445,69 \pm 1,03$	$445,69 \pm 1,05$
Popolazione media in coda [job]	$0,588368 \pm 0,009693$	$0,218968 \pm 0,004846$	$0,033849 \pm 0,001400$	$0,000357 \pm 0,000181$
Popolazione media nel sistema [job]	$0,736091 \pm 0,011160$	$0,365510 \pm 0,007062$	$0,181737 \pm 0,004456$	$0,147698 \pm 0,003673$

A. Invarianza del lavoro totale

La prima osservazione riguarda l'utilizzazione. I quattro valori misurati, moltiplicati per il rispettivo numero di serveri, danno $c\rho = 0,1477, 0,1465, 0,1479$ e $0,1473$: un valore **costante entro l'1%**. Ciò è esattamente quanto previsto dalla (5), poiché il carico di lavoro totale offerto al nodo, $X_{\text{nodo}}E[S] = 0,1472$ erlang, non dipende dal numero di serveri che lo smaltiscono. L'accordo fra il valore teorico e i quattro valori misurati costituisce un ulteriore controllo trasversale della correttezza del modello, questa volta esteso a tutte le configurazioni e non alla sola nominale.

Ne discende immediatamente che aumentando c non si riduce il *lavoro*, ma soltanto la *contesa* per eseguirlo. È una precisazione che orienta tutta l'analisi successiva: qualunque

miglioramento del tempo di risposta dovrà provenire dalla riduzione dell'attesa, non del servizio.

Coerentemente, il tempo di servizio dei job *passed* è identico nelle quattro configurazioni fino alla seconda cifra decimale ($445,68$ – $445,69$ s), con intervalli di confidenza completamente sovrapposti. La variabilità apparente che si osserva nel corrispondente grafico non è un effetto del numero di serveri ma puro rumore campionario, il cui ordine di grandezza ($\pm 1,05$ s) è correttamente descritto dall'intervallo di confidenza. Interpretare quel grafico come evidenza di una dipendenza del servizio da c sarebbe un errore.

B. Scomposizione del tempo di risposta

Il tempo di risposta di un job, misurato dalla sua creazione all'uscita definitiva dal sistema, si scompone in tre contributi

Transitorio - Tempo di servizio passed (media cumulata)

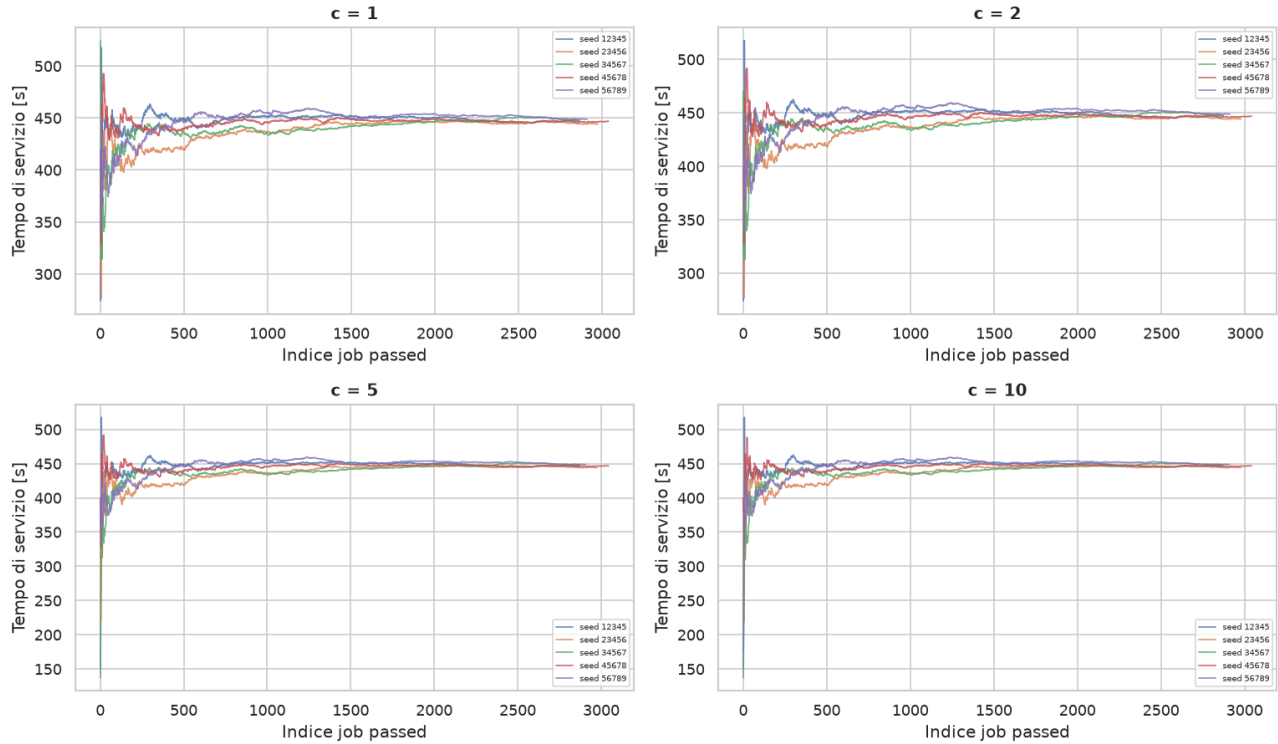


Figura 12. Analisi del transitorio del tempo di servizio dei job passed. L'ascissa è l'indice progressivo del job. La convergenza è pressoché immediata e identica per ogni c , come atteso per una grandezza indipendente dallo stato del sistema.

Tabella XV

SCOMPOSIZIONE DEL TEMPO MEDIO DI RISPOSTA [s] E QUOTE PERCENTUALI, OTTENUTA DALLE TRACCE PER JOB CON LA MEDESIMA PROCEDURA DI BATCH MEANS DELLA TABELLA XIV; LE TRE COMPONENTI SOMMANO A R PER COSTRUZIONE

c	R	attesa	servizio	feedback	quote
1	2662,83	1791,17	447,68	423,98	67,3% att. 15,9% fb.
2	1529,75	658,05	447,70	424,00	43,0% att. 27,7% fb.
5	972,93	101,45	447,55	423,93	10,4% att. 43,6% fb.
10	873,05	1,47	447,63	423,95	0,2% att. 48,6% fb.

mutuamente esclusivi: il tempo complessivamente trascorso in coda, il tempo complessivamente trascorso in servizio e il tempo trascorso nel nodo di revisione umana. La Tabella XV riporta tale scomposizione, ottenuta direttamente dalle tracce per job al netto del periodo di riscaldamento.

I dati mostrano con chiarezza che le componenti di servizio e di feedback sono **rigorosamente invarianti** rispetto a c , come devono essere: nessuna delle due dipende dalla contesa per i serventi. La componente di servizio, 447,55–447,70 s, coincide con il prodotto $v \cdot E[S] = 1,20079 \cdot 372,98 = 447,87$ s previsto dalla teoria; la componente di feedback vale 423,93 s a $c = 5$ contro un valore teorico di $v(p_{\text{failed}} + p_{\text{errored}})E[D] = 0,258 \cdot 1545,4 = 398,7$ s. Lo scarto di +25,2 s è imputabile alla coda pesante della lognormale di ritardo, ed è di poco

superiore a un errore standard: l'intervallo di confidenza al 95% della sola componente di feedback, calcolato con la stessa procedura di batch means, vale infatti $\pm 44,30$ s, da cui un errore standard di 22,1 s e uno scarto pari a 1,14 errori standard.

L'intera variazione del tempo di risposta al variare di c è dunque assorbita dalla sola componente di attesa, che passa da 1791 s a 1,47 s, ossia si riduce di tre ordini di grandezza.

C. L'origine dell'attesa non è il carico

Si giunge così al risultato più istruttivo dello studio. Il sistema opera a utilizzazione $\rho = 0,0296$ nella configurazione nominale, e a $\rho = 0,148$ persino con un solo servente. La teoria delle code classica, applicata a un sistema con arrivi *singoli* e lo stesso carico offerto, predirebbe attese del tutto trascurabili. Il confronto è riportato nella Tabella XVI.

La discrepanza è di due, tre e più ordini di grandezza. **L'attesa osservata non è prodotta dal carico del sistema, ma dalla struttura a gruppi degli arrivi:** quando arriva una build matrix da otto job e i serventi disponibili sono cinque, tre job devono attendere il completamento dei primi, indipendentemente dal fatto che il sistema fosse completamente vuoto un istante prima.

Questa interpretazione è quantificabile in modo esatto grazie alla scomposizione (12) già impiegata in fase di verifica.

Transitorio - Tempo di risposta (media cumulata)

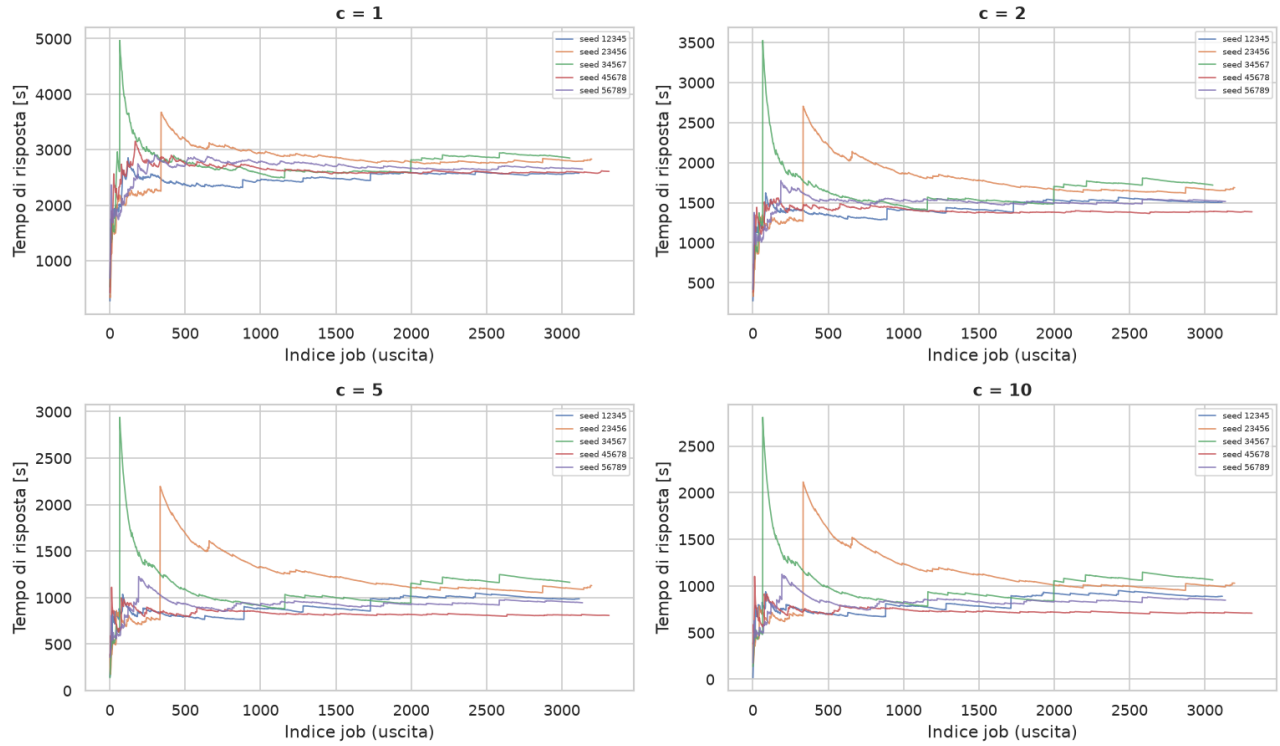


Figura 13. Analisi del transitorio del tempo di risposta al variare del numero di server. Si osservi la maggiore dispersione fra repliche rispetto alle altre due grandezze, dovuta alla forte variabilità del ritardo di feedback umano.

Tabella XVI
ATTESA MEDIA PREVISTA DAL MODELLO $M/M/c$ CON LO STESSO CARICO OFFERTO ($a = 0,1472$ ERLANG) CONTRO ATTESA EFFETTIVAMENTE MISURATA

c	W_q da $M/M/c$ [s]	W_q misurato [s]	rapporto
1	64,38	1791,17	$28\times$
2	2,03	658,05	$324\times$
5	$3,9 \cdot 10^{-5}$	101,45	$2,6 \cdot 10^6\times$
10	$4,36 \cdot 10^{-14}$	1,47	—

Il secondo termine di quella formula si riscrive, ponendo $E[B^2] = E[B]^2 (1 + C_B^2)$,

$$W_q^{\text{gruppo}} = \frac{E[S]}{2} (E[B] (1 + C_B^2) - 1) = 372,98 \cdot 3,4537 = 1288,2 \text{ s} \quad (14)$$

e rappresenta esattamente l'attesa dovuta alla serializzazione dei compagni di gruppo. Esso è **indipendente dal carico**: non contiene λ e non si annulla al tendere a zero dell'utilizzazione. Con un solo server costituisce da solo il **72%** dell'attesa misurata di 1791 s.

La forma (14) ha il pregio di separare i due fattori che generano l'attesa di gruppo — la taglia della build matrix e la sua dispersione — e il loro peso relativo è istruttivo. Con $E[B] = 7,145$ e $C_B^2 = 0,107$, se la build matrix avesse

dimensione *deterministica* pari alla sua media, ossia $C_B^2 = 0$, il termine varrebbe ancora 1146 s. Dei 1288 s complessivi, dunque, **l'89% è prodotto dalla dimensione media della matrice e soltanto l'11% dalla sua variabilità**. Non è l'irregolarità della build matrix a generare l'attesa, ma la sua taglia: un progetto che lanciasse sempre esattamente sette job per build osserverebbe un'attesa di gruppo inferiore di appena del 13%. Il punto rafforza il criterio di dimensionamento della §XI-D, perché sposta il parametro di progetto da una proprietà di dispersione, che lo sviluppatore non controlla direttamente, a una grandezza leggibile per ispezione dal file `.travis.yml`.

La Tabella XVII, ottenuta applicando la (12) al caso degenerare già verificato in §VII-E, mostra come le due componenti si ripartiscano al variare dell'utilizzazione.

Ne risulta che, come ci si attende, in regime di traffico intenso il contributo del carico domina e il termine di gruppo diventa marginale. Ma nel regime in cui il sistema reale effettivamente opera — ρ compreso fra 0,015 e 0,148 — la situazione è rovesciata: quasi tutta l'attesa è generata dalla struttura degli arrivi. Un modo equivalente e ancora più diretto di esprimerlo è confrontare la $M^{[B]}/G/1$ con una $M/G/1$ soggetta allo stesso tasso di job ma con arrivi singoli: a $\rho = 0,1$ la prima produce un'attesa di 1744 s contro i 33 s della seconda, ossia **53 volte maggiore**.

Batch means - Utilizzazione

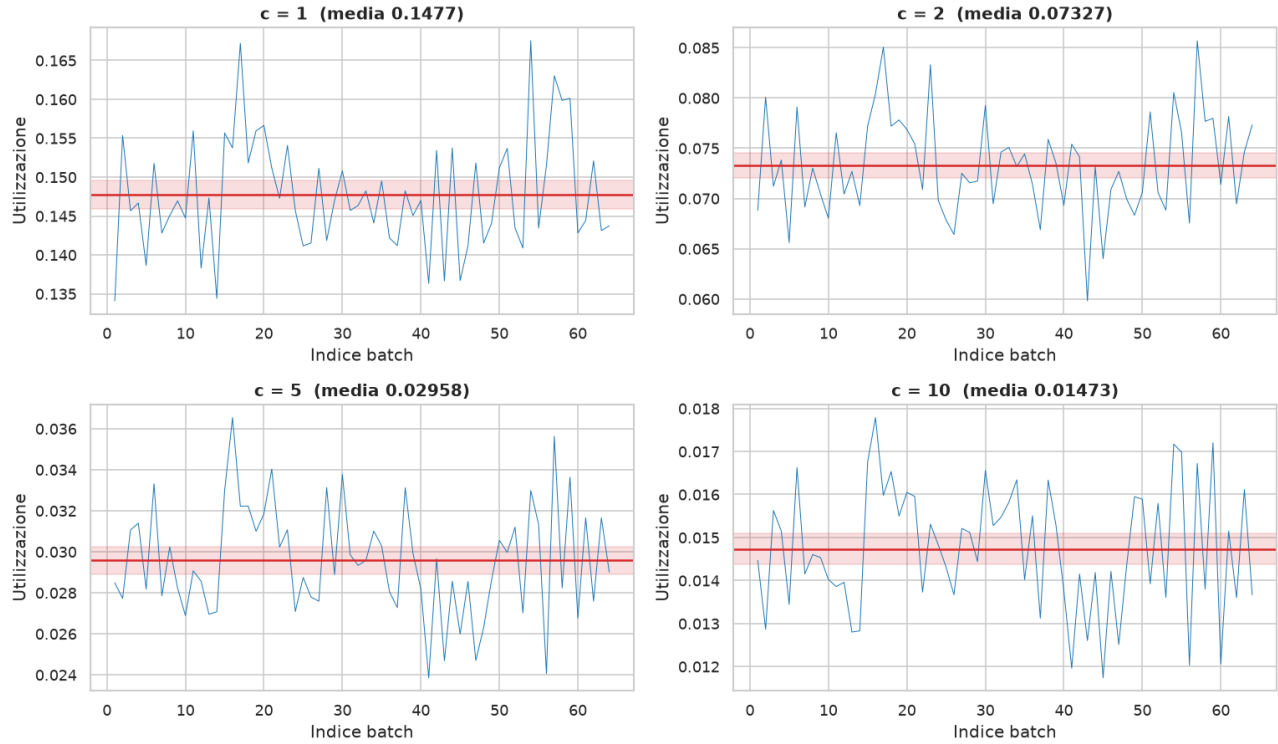


Figura 14. Diagramma diagnostico dei batch means per l'utilizzazione.

Tabella XVII

SCOMPOSIZIONE ESATTA DELL'ATTESA NELLA $M^{[B]}/G/1$: CONTRIBUTO DEL CARICO CONTRO CONTRIBUTO DELLA STRUTTURA A GRUPPI

ρ	W_q tot. [s]	carico [s]	gruppo [s]	quota gruppo
0,1	1744,4	204,0	1540,4	88,3%
0,3	2327,1	786,7	1540,4	66,2%
0,5	3376,0	1835,6	1540,4	45,6%
0,7	5823,5	4283,1	1540,4	26,5%
0,9	18060,8	16520,3	1540,4	8,5%

Questo risultato ha una conseguenza metodologica generale, che va oltre il caso di studio: **modellare una pipeline CI/CD come un sistema ad arrivi singoli condurrebbe a conclusioni radicalmente errate**, facendo apparire il dimensionamento del pool di runner come una questione irrilevante quando invece non lo è. È la struttura a build matrix, non il volume di traffico, a determinare la latenza percepita.

D. Rendimenti decrescenti e il ginocchio della curva

Le riduzioni marginali del tempo di risposta sono riportate nella Tabella XVIII.

Il beneficio per servente aggiunto cala di un fattore 6 nel primo passaggio e di un ulteriore fattore 9 nel secondo: si tratta di **rendimenti fortemente decrescenti**. Ma il dato più significativo non è la decrescita in sé, bensì la posizione del ginocchio della curva.

Tabella XVIII

BENEFICIO MARGINALE DELL'AGGIUNTA DI SERVENTI

Transizione	ΔR [s]	riduz.	ΔR per servente
$c : 1 \rightarrow 2$	1133,1	42,6%	1133,1 s
$c : 2 \rightarrow 5$	556,8	36,4%	185,6 s
$c : 5 \rightarrow 10$	99,9	10,3%	20,0 s

L'attesa a $c = 10$ vale 1,47 s, ossia è praticamente annullata; a $c = 5$ vale ancora 101 s. Il motivo è strutturale e discende direttamente dalla §XI-C: la dimensione massima di gruppo osservata nel progetto è esattamente 10. Con $c \geq 10$ ogni job di ogni build matrix trova sempre un servente libero al proprio arrivo (al netto di job già presenti in coda per via del feedback) e la serializzazione intra-gruppo, che come si è visto costituisce la quasi totalità dell'attesa, scompare. Con $c = 5$, invece, le build da più di 5 job — che rappresentano l'81,7% del totale — eccedono la capacità del pool e generano attesa.

Se ne trae un criterio di dimensionamento operativo e non ovvio: **in un sistema di integrazione continua a basso carico il pool di runner va dimensionato sulla dimensione della build matrix, non sul tasso di arrivo**. Il parametro rilevante è la distribuzione di B , non λ . Un dimensionamento basato sull'utilizzazione — che a $c = 5$ vale il 3% e suggerirebbe di ridurre il pool — porterebbe a una decisione sbagliata.

Batch means - Tempo medio di risposta [s]

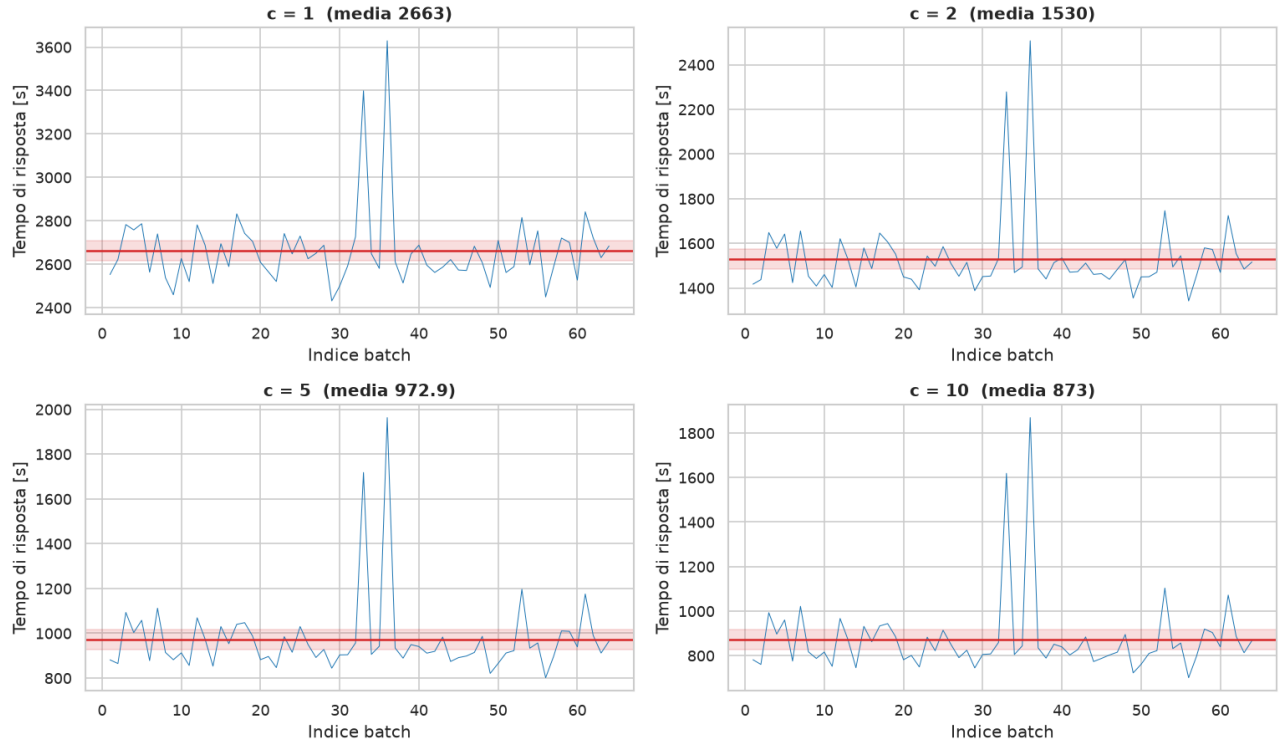


Figura 15. Diagramma diagnostico dei batch means per il tempo di risposta. I picchi in corrispondenza degli indici 33–36, identici in tutti e quattro i pannelli, sono prodotti dal medesimo evento di feedback eccezionalmente lungo.

E. Il collo di bottiglia è la revisione umana

La scomposizione della Tabella XV conduce a una seconda conclusione decisionale. Poiché le componenti di servizio e di feedback sono invarianti rispetto a c , il tempo di risposta ammette un **limite inferiore invalicabile**:

$$R_{\min} = v E[S] + v (p_{\text{failed}} + p_{\text{errored}}) E[D] \approx 447,9 + 398,7 = 846,6 \text{ s} \quad (15)$$

Il valore misurato a $c = 10$, pari a 873,05 s, lo approssima entro il 3,2%: la configurazione a dieci serventi ha già *sostanzialmente esaurito* il margine di miglioramento ottenibile aggiungendo infrastruttura. Portare c a 20 o a 100 non produrrebbe alcun beneficio apprezzabile.

Al limite (15), la revisione umana contribuisce per il 47,1%, ossia **più del servizio stesso**. Il collo di bottiglia del sistema, una volta rimossa la contesa per i runner, non è l'infrastruttura di calcolo ma il tempo di reazione dello sviluppatore. Ne segue che, oltre $c = 10$, ogni ulteriore investimento andrebbe indirizzato non all'acquisto di runner ma alla riduzione di $E[D]$ — per esempio migliorando la notifica dei fallimenti — oppure alla riduzione della probabilità di rientro q , agendo sulla qualità dell'infrastruttura per abbattere la frazione di errored, che da sola genera il 93,1% del traffico di ritorno: dei $q = 0,1672$ di probabilità di rientro, 0,1557 provengono dagli errored e appena 0,0115 dai failed. Si osservi

che, per la (4), una riduzione di q agisce simultaneamente su entrambi i termini della (15).

F. Precisione delle stime e numeri casuali comuni

Un ultimo aspetto merita commento, perché riguarda l'affidabilità delle conclusioni precedenti.

Si osservi nella Tabella XIV che la semi-ampiezza dell'intervallo di confidenza sul tempo di risposta vale circa 44 s in tutte le configurazioni, dalla prima all'ultima, mentre quella sul tempo di attesa scende da 14,1 s a 0,17 s. In termini relativi ciò significa che il tempo di risposta è stimato con precisione dell'1,7% a $c = 1$ e del 5,1% a $c = 10$: la precisione *peggiora* all'aumentare dei serventi.

La spiegazione segue dalla scomposizione della Tabella XV. La varianza del tempo di risposta è dominata dalla componente di feedback, la cui distribuzione lognormale ha coefficiente di variabilità $C_D^2 = 96,4$ contro lo 0,575 del servizio, ed è per costruzione indipendente da c . Il divario è ancora più netto sulle varianze assolute, che sono le grandezze che effettivamente si sommano nel tempo di risposta: tenuto conto delle rispettive medie ($E[D] = 1545$ s contro $E[S] = 373$ s) si ottiene $\text{Var}[D]/\text{Var}[S] \approx 2900$. All'aumentare dei serventi la componente di attesa — l'unica che varia — si riduce, ma la componente rumorosa resta invariata: il rapporto segnale-rumore peggiora.

Batch means - Tempo medio di attesa [s]

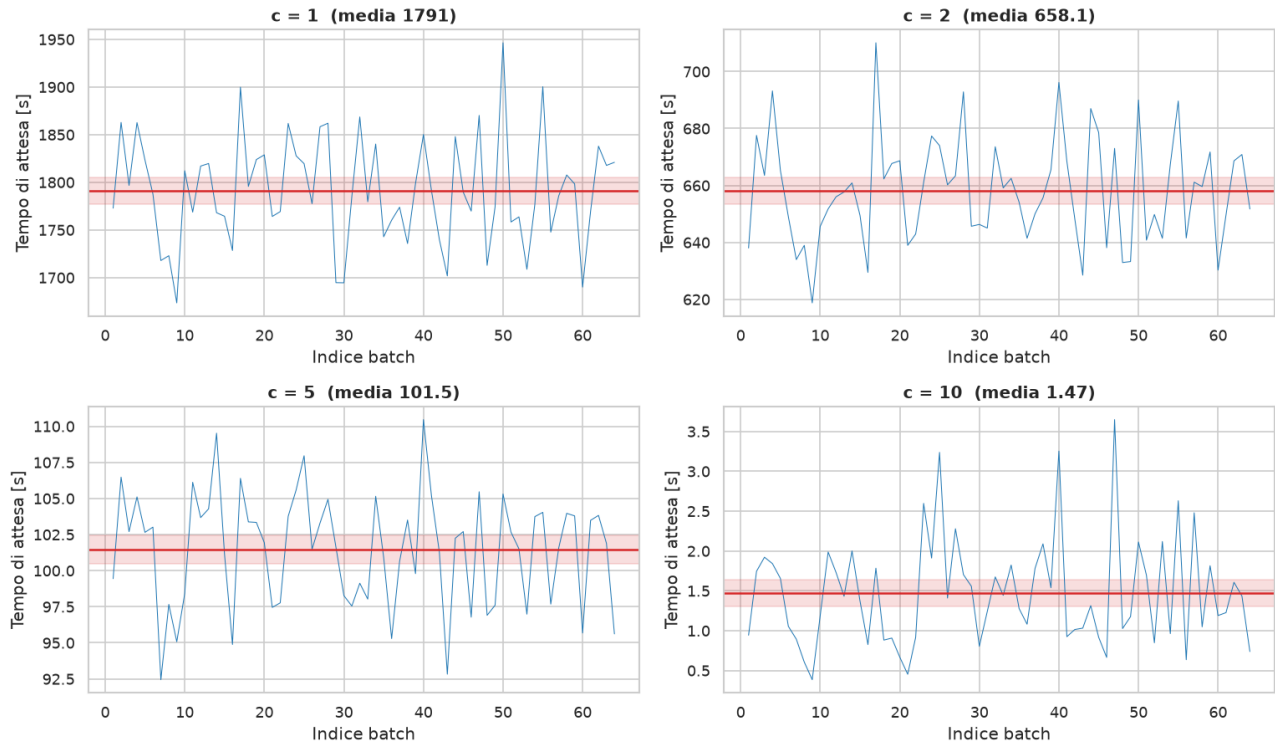


Figura 16. Diagramma diagnostico dei batch means per il tempo di attesa in coda. Si noti il cambiamento di scala fra i pannelli: da circa 1 800 s per $c = 1$ a circa 1,5 s per $c = 10$.

Applicando la procedura di batch means alle singole componenti della Tabella XV l'argomento cessa di essere qualitativo. A $c = 5$ l'intervallo di confidenza al 95% vale $\pm 0,98$ s sulla componente di attesa e $\pm 1,09$ s su quella di servizio, ma $\pm 44,30$ s su quella di feedback — a fronte di un $\pm 44,17$ s sul tempo di risposta complessivo. *La totalità dell'incertezza su R proviene dunque dal ritardo di revisione umana*: attesa e servizio vi contribuiscono in misura trascurabile. **La precisione della stima del tempo di risposta è limitata dalla variabilità del comportamento umano, non da quella del sistema di calcolo**, ed è questo il motivo per cui nessun allungamento ragionevole della run potrebbe ridurre sensibilmente quell'intervallo.

Vi è però una circostanza che rende i confronti fra configurazioni assai più affidabili di quanto i singoli intervalli di confidenza suggerirebbero. Le quattro configurazioni sono simulate a partire dal **medesimo seme** e, come descritto in §VI, ogni sorgente stocastica dispone di uno stream dedicato. Ne consegue che la sequenza dei tempi di servizio, degli esiti, dei ritardi di feedback e delle dimensioni dei gruppi è *identica* nelle quattro configurazioni: cambia soltanto l'ordine in cui i job vengono serviti. Si tratta della tecnica di riduzione della varianza dei *numeri casuali comuni*, ottenuta qui come conseguenza diretta della disciplina di assegnazione degli stream.

L'evidenza empirica è nella Tabella XV: le componenti di servizio (447,55–447,70 s) e di feedback (423,93–424,00 s) coincidono entro 0,15 s, ossia lo 0,03%, in tutte e quattro le configurazioni. I picchi anomali nei diagrammi dei batch (Fig. 15) compaiono negli stessi indici di batch per ogni c , il che ne conferma l'origine comune. Poiché la componente di rumore è la medesima, le *differenze* fra configurazioni sono note con precisione assai maggiore dei singoli valori, e le riduzioni riportate nella Tabella XVIII sono da considerarsi significative nonostante la parziale sovrapposizione degli intervalli di confidenza sul tempo di risposta.

G. Sintesi decisionale

Riassumendo il quadro dal punto di vista di uno sviluppatore che debba scegliere il piano di servizio:

- il **passaggio da 1 a 2 runner** è il più redditizio in assoluto: dimezza quasi il tempo di risposta al costo di un solo servente aggiuntivo;
- la **configurazione a 5 runner**, storicamente offerta gratuitamente ai progetti open-source, rappresenta un compromesso ragionevole: ha già eliminato il 94% dell'attesa rispetto al servente singolo, e residua un'attesa media di 101 s;
- il **passaggio a 10 runner** azzerà di fatto l'attesa, ma migliora il tempo di risposta di appena il 10%, poiché

Batch means - Popolazione media nel sistema [job]

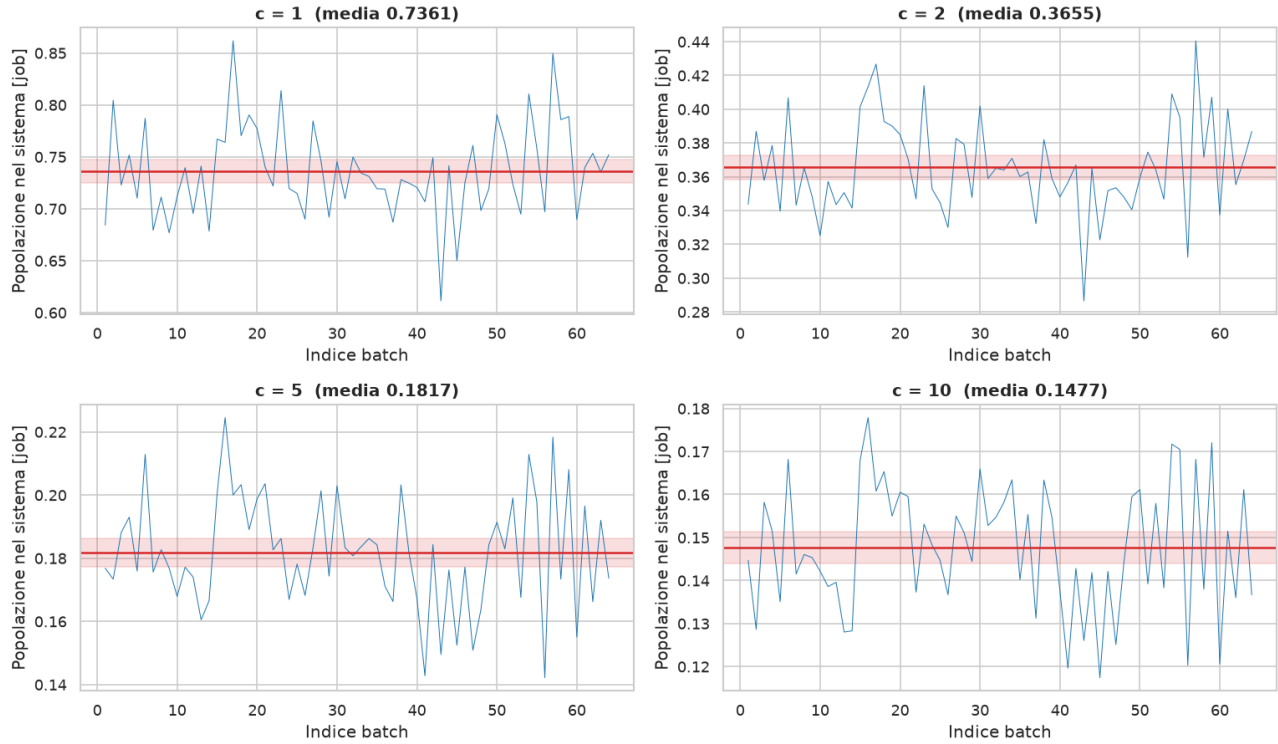


Figura 17. Diagramma diagnostico dei batch means per la popolazione media nel nodo (coda più serventi).

il tempo di risposta è ormai dominato da servizio e revisione umana. Il suo valore va giudicato in rapporto al costo del raddoppio del pool, e dipende da quanto la riduzione di 100 s sia critica per il flusso di lavoro del progetto;

- **oltre 10 runner non vi è alcun beneficio**, perché si è raggiunto il limite (15) imposto dal servizio e dal feedback.

XII. CONCLUSIONI

Si è sviluppato e collaudato un simulatore a eventi discreti di una pipeline CI/CD di tipo Travis CI, formalizzata come rete di code $M^{[B]}/G/c$ con anello di retroazione umana, calibrata su 16 278 job reali del progetto SonarSource/sonar-java estratti dal dataset TravisTorrent.

Sul piano metodologico, il contributo principale consiste nelle fasi di verifica e validazione. La verifica è articolata su cinque livelli e 70 controlli automatici, il più stringente dei quali confronta il motore con la soluzione analitica esatta di una coda $M^{[B]}/G/1$ su cinque livelli di utilizzazione compresi fra 0,1 e 0,9: sul tempo medio di attesa, in tutti i casi l'intervallo di confidenza costruito sulla simulazione contiene il valore teorico, con errore relativo inferiore all'1% fino a $\rho = 0,7$. La validazione confronta esclusivamente grandezze misurate dalle tracce con grandezze misurate sul dataset, evitando la circolarità insita nel confronto fra parametri di

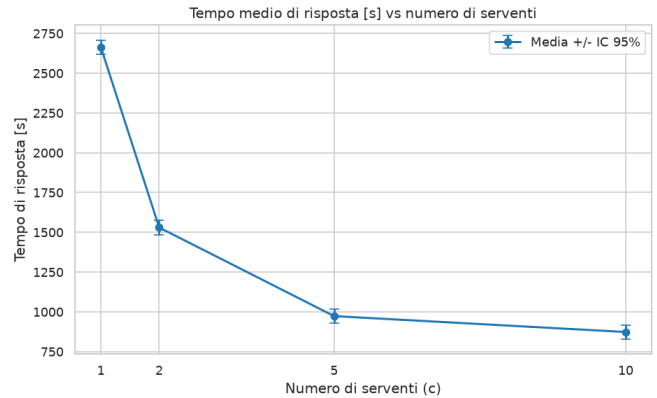


Figura 18. Variazione del tempo medio di risposta al variare del numero di serventi. Si osservi l'appiattimento della curva oltre $c = 5$, che riflette il raggiungimento del limite inferiore imposto da servizio e revisione umana.

ingresso e dati di calibrazione — con la sola parziale eccezione del tasso di arrivo, il cui limitato potere validante è dichiarato in §VIII-E; laddove si osservano residui sistematici, essi sono stati scomposti analiticamente nelle loro cause, che risultano tutte identificate e riconducibili a scelte di stima note e non a errori di implementazione.

Sul piano dei risultati, lo studio conduce a tre conclusioni che l'analisi superficiale delle sole medie non avrebbe rivelato.

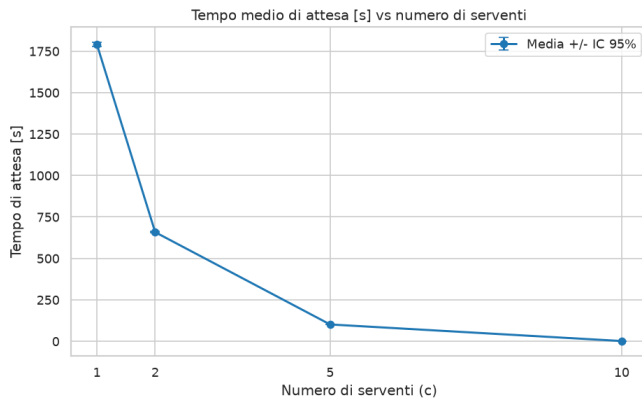


Figura 19. Variazione del tempo medio di attesa al variare del numero di serveri. La caduta di tre ordini di grandezza fra $c = 1$ e $c = 10$ è prodotta dalla progressiva eliminazione della serializzazione intra-gruppo.

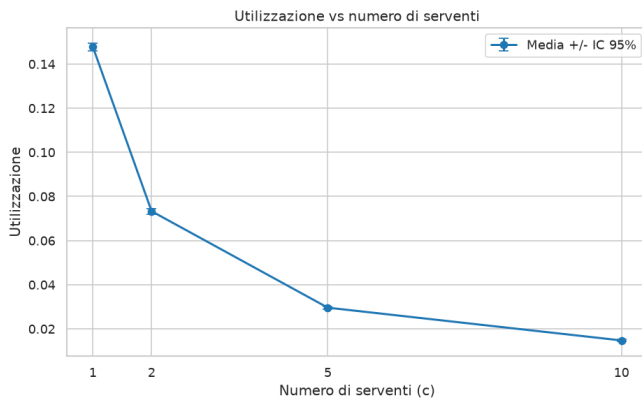


Figura 20. Variazione dell'utilizzazione al variare del numero di serveri. L'andamento è esattamente proporzionale a $1/c$, poiché il carico di lavoro totale non dipende dal dimensionamento del pool.

La prima è che **l'attesa in coda osservata non è prodotta dal carico**. Il sistema opera a utilizzazione compresa fra l'1,5% e il 15%, regime in cui un modello ad arrivi singoli predirebbe attese trascurabili; l'attesa effettivamente misurata eccede tale previsione di due o tre ordini di grandezza. La causa è la struttura a build matrix degli arrivi: la scomposizione esatta del tempo di attesa nella $M^{[B]}/G/1$ mostra che, nel regime operativo reale, fino al 72% dell'attesa è generata dalla serializzazione dei job appartenenti alla stessa build e non dalla contesa con altre build.

La seconda è il criterio di dimensionamento che ne discende: **il pool di runner va dimensionato sulla distribuzione della dimensione della build matrix, non sul tasso di arrivo**. Il ginocchio della curva prestazionale cade esattamente in corrispondenza della dimensione massima di build osservata, $c = 10$, dove l'attesa media crolla a 1,5 s.

La terza è l'individuazione del collo di bottiglia effettivo. Poiché le componenti di servizio e di revisione umana del tempo di risposta sono invarianti rispetto al numero di serveri, esiste un limite inferiore di circa 850 s che nessun

potenziamento dell'infrastruttura può superare, e al quale la revisione umana contribuisce per il 47,1% — più del servizio stesso. Oltre i dieci runner, ogni ulteriore investimento va indirizzato alla riduzione del ritardo di notifica dei fallimenti o alla riduzione della frazione di job `errored`, non all'acquisto di capacità di calcolo.

Da un punto di vista pratico, uno sviluppatore di progetti open-source che avesse necessità di continuare a utilizzare Travis CI nonostante i costi dispone quindi di un criterio quantitativo per la scelta del piano: il passaggio da uno a due runner è di gran lunga il più redditizio, la configurazione a cinque runner rappresenta un compromesso ragionevole, e il raddoppio a dieci va valutato in rapporto al costo poiché produce un miglioramento di appena il 10% del tempo di risposta complessivo.

Sviluppi futuri

Tre direzioni appaiono naturali. La prima è la rimozione dell'ipotesi di arrivi poissoniani omogenei, sostituendola con un processo modulato che riproduca i cicli giorno/notte e settimanali osservati nei dati, così da quantificare i picchi di coda che il modello omogeneo per costruzione non cattura. La seconda è l'introduzione di una correlazione fra i tempi di servizio dei job appartenenti alla stessa build matrix, ipotesi plausibile e potenzialmente rilevante poiché aumenterebbe la variabilità del carico portato da ciascun gruppo. La terza è uno studio di sensibilità rispetto al tasso di arrivo, che porti il sistema verso la saturazione: il motore è già stato collaudato fino a $\rho = 0,9$ in fase di verifica, e in quel regime il modello a code produrrebbe informazione decisionale di natura diversa da quella qui ottenuta.

APPENDICE A RIPRODUCIBILITÀ

L'intera analisi è riproducibile eseguendo due comandi (vedi `README.md`). Il primo esegue l'analisi della configurazione nominale, comprensiva di transitorio, regime, verifica su cinque livelli e validazione; il secondo esegue lo studio what-if sul numero di serveri. I parametri del modello risiedono in `config.json`, l'elenco delle configurazioni da confrontare in `experiments.json` e i parametri della verifica strutturale in `verification.json`. Nessun parametro è inserito nel codice sorgente. Tutti i report tabellari citati nel presente documento sono prodotti automaticamente in formato CSV nella cartella di uscita, oppure stampati in output durante l'esecuzione.

RIFERIMENTI BIBLIOGRAFICI

- [1] L. M. Leemis and S. K. Park, *Discrete-Event Simulation: A First Course*. Upper Saddle River, NJ: Pearson Prentice Hall, 2006.
- [2] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for full-stack research on continuous integration," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 447–450.
- [3] M. L. Chaudhry and J. G. C. Templeton, *A First Course in Bulk Queues*. New York: Wiley, 1983.
- [4] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York: Wiley-Interscience, 1975.

- [5] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York: McGraw-Hill, 2015.
- [6] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [7] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find," *Commun. ACM*, vol. 31, no. 10, pp. 1192–1201, 1988.